

Randomized Agreement, Verifiable Secret Sharing and Multi-Party Computation in Granular Synchrony

Ananya Appan, David Heath, and Ling Ren

University of Illinois Urbana-Champaign
{aappan2, daheath, renling}@illinois.edu

Abstract. Granular Synchrony (Giridharan et al. DISC 2024) is a new network model that unifies the classic timing models of synchrony and asynchrony. The network is viewed as a graph consisting of a mixture of synchronous, eventually synchronous, and asynchronous communication links. It has been shown that Granular Synchrony allows deterministic Byzantine agreement protocols to achieve a corruption threshold in between complete synchrony and complete asynchrony if and only if the network graph satisfies the right condition, namely, that no two groups of honest parties of size $n - 2t$ can be partitioned from each other.

In this work, we show that the same network condition is also tight for Agreement on a Common Subset (ACS), Verifiable Secret Sharing (VSS), and secure Multi-Party Computation (MPC) with guaranteed output delivery, when the corruption threshold is between one-third and one-half. Our protocols are randomized and assume that all links are either synchronous or asynchronous. Our ACS protocol incurs an amortized communication cost of $O(n^3\lambda)$ bits per input, and our VSS and MPC protocols incur amortized communication costs of $O(n^3)$ and $O(n^4)$ field elements per secret and per multiplication gate, respectively. To design our protocols, we also construct protocols for Reliable Broadcast and Externally Valid Byzantine Agreement (EVBA), which are of independent interest.

1 Introduction

Distributed Byzantine fault tolerant or cryptographic protocols involve n mutually distrusting parties, a fraction of whom can be corrupted. Examples include Byzantine Agreement (BA), Verifiable Secret Sharing (VSS), and secure Multi-Party Computation (MPC). A BA protocol allows honest parties with different input bits to agree on a common output bit [LSP82]. A VSS protocol allows a dealer to share a secret with a group of n parties, so that a unique secret can be reconstructed later and corrupt parties alone cannot reconstruct the secret [CGMA85, Fel87]. An MPC protocol allows n parties to jointly compute a function of their private inputs without revealing their individual inputs [Yao86, GMW87].

The communication network is a crucial aspect of the model in distributed computing. In a *synchronous* network, each message is delivered within a known bounded time. *Asynchronous* networks are weaker, and only guarantee that each message will be delivered *eventually*, after a potentially unbounded delay. Both network models have their advantages and drawbacks. Synchronous networks tolerate more corruptions: up to half of the parties can be malicious (also called Byzantine); asynchronous (or partially synchronous [DLS88]) networks allow only up to one-third malicious parties. However, synchronous networks make a strong assumption about message delays, and asynchronous (or partially synchronous) networks are thought to be more realistic.

In light of the above trade-offs, a natural question is whether and how we can tolerate more than one-third corruptions without resorting to a fully synchronous network. There have been many such attempts, most of which target easier faults than Byzantine [LM07, GPS19] or make additional assumptions such as trusted hardware [CMSK07, VCB⁺11].

Recently, the work of Giridharan et al. [GAC⁺24] introduced a new network model called *granular synchrony* that allows higher than one-third corruptions. The granular synchrony model assumes the network consists of a mixture of synchronous and asynchronous links. Essentially, it contends that while full synchrony is too strong an assumption, complete asynchrony is overly pessimistic because it is unlikely that an adversary can delay messages on all links in the entire network. [GAC⁺24] presents deterministic protocols for Byzantine agreement in the granular synchrony model that provides a full-spectrum trade-off between network assumptions and Byzantine fault tolerance: the protocols tolerate anywhere between one-half and one-third Byzantine faults depending on how synchronously connected the parties are.

In this work, we present protocols for VSS, randomized agreement, and MPC that exhibit a similar full-spectrum trade-off between fault tolerance and network assumption in the granular synchrony model. Next, we describe the granular synchrony model and state our contributions.

1.1 Granular Synchrony

In granular synchrony, there is a bidirectional communication link between every pair of parties, and the link is either synchronous or asynchronous.¹ We emphasize that parties are *unaware* of which links are synchronous and which links are asynchronous. It is also assumed that communication links are *static*, i.e., they remain synchronous or asynchronous throughout the protocol.

Two groups of parties are said to be *partitioned* if there is *no* path of synchronous links from any party in one group to any party in the other group. In [GAC⁺24], the authors design a protocol for Byzantine agreement, under the assumption that no two groups of honest parties that are each of size $\geq n - 2t$ are partitioned from each other.

When $n = 2t + 1$, the above condition enforces that no two honest parties are partitioned and degenerates to synchrony. When $n = 3t + 1$, there simply cannot be two disjoint sets of $n - 2t$ honest parties, and the model degenerates to complete asynchrony. What is interesting about the granular synchrony model is that it captures every possible fault tolerance (t/n) *in between* one-half and one-third, and completely characterizes the network condition needed to tolerate that fraction of corruption for Byzantine agreement – naturally, it is a network condition in between synchrony and asynchrony. For instance, if $t = 0.4n$, the condition is that no two groups of honest parties of size $0.2n$ are partitioned from each other. This allows, for example, two groups of honest parties of sizes $0.18n$ and $0.42n$, or three groups of honest parties of sizes $0.09n$, $0.09n$, and $0.42n$, to be partitioned from each other.

1.2 Our contributions

We present the first protocols for randomized Byzantine agreement (including Externally Valid Byzantine Agreement and Agreement on a Common Subset), VSS, and MPC in the granular synchrony model. To construct our protocols, we also design a Reliable Broadcast (RBC) protocol that allows parties to agree on a message sent by a designated sender. Like [GAC⁺24], we assume that no two groups of honest parties of size $\geq n - 2t$ can be partitioned from each other, and we show that this assumption is necessary. Table 1 lists our protocols, along with their associated costs and required assumptions.

¹ The work of [GAC⁺24] focuses on deterministic agreement protocols and hence also considers *eventually synchronous* links. Our work studies randomized protocols, so we replace all the eventually synchronous links with asynchronous links.

	Primitives Used	Communication (bits)	Assumption	Section
RBC	—	$O(n^2 v + n^4\lambda)$	PKI, CRH	§5
VSS	RBC	$O(n^3 v \cdot L + n^4\lambda)$	PKI, CRH, DL	§6
EVBA^{†*}	RBC	$O(n^5\lambda)$	PKI	§7
ACS[†]	RBC, VSS, EVBA	$O(n^3\lambda v + n^6\lambda)$	PKI, RO, DDH	§7
MPC[†]	VSS, ACS	$O(n^4 v \cdot c_M + n^6\lambda)$	PKI, RO, DDH	§8

[†] Communication *in expectation*.

* EVBA assumes external common coins. While our ACS protocol uses EVBA, it does not require this assumption.

– Notation: $|v|$: size of the input, L : number of secrets being shared, λ : computational security parameter, c_M : number of multiplication gates in the MPC circuit.

– Abbreviations: PKI: Public Key Infrastructure, CRH: Collision Resistant Hash, RO: Random Oracle, DL: Hardness of the Discrete Logarithm problem, DDH: Hardness of the Decisional Diffie-Hellman problem.

Table 1: Our protocols, along with their communication costs and assumptions. When we assume PKI, we additionally assume digital signatures.

We remark that the focus of [GAC⁺24] was *deterministic* protocols for the Byzantine Agreement problem only. Therefore, it assumes some links between parties are *partially synchronous*, meaning that these links eventually become synchronous. Partial synchrony is one way to circumvent the celebrated FLP impossibility result [FLP85]: it is impossible to achieve deterministic BA in complete asynchrony. Another natural way to circumvent the FLP impossibility result is to design randomized protocols. In this work, we study a wider range of problems, including VSS and MPC, which are inherently randomized. Thus, we assume a weaker variant of granular synchrony where non-synchronous links are *asynchronous* instead.

Our protocols, being randomized, also differ fundamentally from those presented in [GAC⁺24]. As we study a wide range of problems, we take a modular approach. In particular, our approach centers on a building block that we call *Collect*, a protocol primitive that addresses the core challenges of granular synchrony. Our higher level protocols are built on top of this powerful primitive.

We use *Collect* to design protocols for RBC and VSS. We then construct an Externally Valid Byzantine Agreement (EVBA) protocol using RBC as a black box; this reduction works even in an *asynchronous network* with $n > 2t$. We then construct Agreement on Common Subset (ACS) from EVBA using the standard template of [BOCG93]. Lastly, we present an MPC protocol in the granular synchrony model using VSS and ACS. Table 1 summarizes our new results.

1.3 Overview

The granular synchrony model calls for completely new techniques in protocol design than those used for fully synchronous and fully asynchronous networks. Protocols designed for synchronous networks proceed in rounds, with each round taking Δ time. In each round, a party is guaranteed to receive all messages sent by each honest party in that round. In asynchronous networks, there is no bound on message delay. Thus, asynchronous protocols are event-based, and each party waits for at most $n - t$ messages before proceeding to the next step. Note that in asynchronous networks, parties cannot wait for more messages to arrive, because the corrupt parties might never send those messages. For many protocols in asynchronous networks, it is crucial that a majority of these $n - t$ messages are from honest parties. This is usually the reason why many problems can be solved in asynchrony only if $n > 3t$ holds.

In granular synchrony, since some communication links may be asynchronous, we cannot rely solely on messages to arrive within a bounded time Δ , as is done in synchrony. At the same time,

waiting for the first $n - t$ messages, as is typical in asynchrony, does not work either. This is because $n > 3t$ need not hold, so there may not be an honest majority among these $n - t$ messages.

As mentioned, we propose a building block that tackles the core challenges of granular synchrony and underlies all our protocols. At a high level, this building block collects messages from a large enough set of parties that contain an honest majority without relying solely on timeouts. This collection of messages has desirable properties that enable us to design all our protocols. We next describe how our building block Π_{Collect} works, and how we use it in higher-level protocols. Throughout our description, for simplicity, we assume that all messages are signed by the sending party.

Our building block Π_{Collect} . Π_{Collect} takes as input one message from each party, and it ensures that each party outputs messages from a set of $n - t$ parties, *together with the messages from all parties with synchronous paths to those $n - t$ parties*. We refer to this set of messages as a *collection*. The key observation underlying Π_{Collect} is that if parties keep relaying each others' inputs for sufficiently many² rounds, they are guaranteed to receive the inputs of all synchronously connected parties. Combining $n - t$ such sets of inputs results in a collection.

A collection offers two useful properties. The first is *honest majority*, i.e., a collection contains messages from at least $t + 1$ honest parties. The second is *honest intersection*, i.e., any two collections must have at least one honest message in common. Honest intersection is reminiscent of the quorum intersection property used in many consensus protocols. Looking ahead, we rely on honest intersection to design our agreement protocols, and we use honest majority to design our VSS protocol.

We first explain why honest majority holds. Note that any collection contains at least $n - 2t$ honest messages. Consider the set of honest parties whose messages are included in the collection. The number of honest parties that *do not* have synchronous paths to parties in this set is less than $n - 2t$, due to the “no partition” assumption made on the communication graph. Since messages from all other honest parties *are* included in the collection, the collection contains more than $(n - t) - (n - 2t) = t$ honest messages.

We next explain why honest intersection holds. Since any collection contains at least $n - 2t$ honest messages, there cannot be two collections whose honest messages are from parties that are partitioned from each other. For every party whose message is included in a collection, the messages of all parties with synchronous paths to it are also included. Thus, any two collections must have an honest message in common.

Protocol composition for Π_{Collect} . Because of our modular approach, we need to address new challenges that arise from protocol composition.

Recall that Π_{Collect} requires a party to receive messages from all synchronously connected parties within a bounded time. For this, we implicitly require all synchronously connected parties to invoke Π_{Collect} within a bounded time of each other, i.e., if some party invokes Π_{Collect} at time T , then all parties synchronously connected to it must invoke Π_{Collect} within time $T + B$, for some known value B . We call this requirement *timely participation*.

Timely participation presents a new challenge to protocol composition. In many cases, timely participation in Π_{Collect} is contingent on the “timely termination” of the preceding (sub-)protocol. Intuitively, this means that if a party completes a (sub-)protocol, then all parties synchronously connected to it need to complete that (sub-)protocol within a bounded time. We ensure timely termination holds in all our protocols, including Π_{Collect} . In Π_{Collect} , this is done by having parties relay the first collection they receive. This also ensures that once some party outputs a collection, parties asynchronously connected to it eventually output a collection.

² This could, for instance, be the diameter of the graph, which never exceeds the number of parties.

Next, we explain how we use Π_{Collect} as a subroutine to build our protocols.

Reliable Broadcast. A reliable broadcast protocol allows parties to agree on a message sent by a (potentially Byzantine) sender. To broadcast a message, the sender first sends the message to all parties. To ensure that agreement holds even if the sender is malicious, parties invoke an instance of Π_{Collect} with the message received from the sender as input (if any). On receiving a collection of messages that are all *equal*, a party can safely output the message. Agreement follows from the honest intersection property of a collection. Timely termination for broadcast follows from the timely termination of Π_{Collect} .

Π_{Collect} incurs an increased communication overhead due to the need to relay messages, including the collection. To broadcast long messages, we reduce this overhead by having parties participate with the *hash* of the sender’s message as input to Π_{Collect} .

Externally Valid Byzantine Agreement. EVBA is used to agree on a bit that satisfies an externally provided predicate. Our EVBA follows the standard template of agreement protocols that use a global strong common coin. Such protocols proceed in iterations where, at the end of an iteration, honest parties have “partial” agreement: they output either bit b or a special value $\perp$. Parties that output $\perp$ toss a coin and use the coin value as their inputs for the next iteration. If all honest parties participate with the same input in an iteration, then they output a common bit and terminate at the end of the iteration. This happens with a probability of $1/2$ every iteration, i.e., if the coin flip matches b . At a high level, partial agreement in each iteration is ensured by quorum intersection. Using RBC, $t < n/2$ suffices, because two quorums of $n - t$ parties intersect at $\geq n - 2t \geq 1$. Even if the intersection is a corrupt party, due to RBC, the *same* message will be present in both quorums.

We then use EVBA to design an Agreement on Common Subset (ACS), a protocol that agrees on a common set of inputs provided by $n - t$ parties, by using the standard template of [BOCG93]. We note that while our EVBA assumes external common coins, our ACS does not as it generates its own common coins using techniques in [DYX⁺22].

Verifiable Secret Sharing. Our VSS protocol uses Shamir secret-sharing to enable a dealer to share points on a d -degree polynomial whose constant term is a secret held by the dealer. The degree d is chosen to be less than $n - t$ so that shares received by honest parties can later be used to reconstruct the secret³. A VSS protocol ensures that honest parties receive shares that lie on the *same* polynomial, even when the dealer is corrupt. To convince parties of this, the dealer uses a polynomial commitment: a cryptographic primitive that allows the dealer to commit to a polynomial without revealing it, and prove to parties that the shares they receive are correct evaluations of the polynomial.

Thus, parties must agree on 1) the polynomial commitment 2) whether all parties have received valid shares that lie on the committed polynomial. To achieve the former, the dealer reliably broadcasts the polynomial commitment. To achieve the latter, parties participate in an instance of Π_{Collect} either with an acknowledgment that they received a valid share, or a complaint against the dealer. Owing to the honest majority of a collection, when the dealer is honest, a collection of inputs contains at least $t + 1$ acknowledgments from honest parties. Using a trick proposed in [DXT⁺23], by setting d to be $n - t$, the dealer can simply broadcast the shares of all parties whose acknowledgments were not included in the collection without revealing the secret, and the validity of these shares can be verified publicly.

³ $d + 1$ points are sufficient to reconstruct a d -degree polynomial

Multi Party Computation. Our MPC protocol evaluates an arithmetic circuit over a field that consists of addition and multiplication gates. Due to the linear nature of Shamir secret-sharing, addition gates can be evaluated locally, without any communication. We evaluate multiplication gates using Beaver’s trick [Bea92] by generating multiplication triples $([a], [b], [ab])$ of *uniformly random* values a and b in an input-independent pre-processing phase

We use the framework of [CP16] to generate multiplication triples. Each party first locally generates and secret shares a multiplication triple. The parties then interact to verify that the shared triples are indeed multiplication triples, and agree on a set $\mathcal{CS}$ of verified triples. Since triples shared by corrupt parties are known to the adversary, the triples in $\mathcal{CS}$ are *combined* using a linear transformation to obtain triples that are uniformly random to the adversary.

This transformation requires $\mathcal{CS}$ to contain triples shared by $t + 1$ *honest* parties. In more detail, triples in $\mathcal{CS}$ are used to define polynomials $A(\cdot)$, $B(\cdot)$ and $C(\cdot)$ of degrees t , t and $2t$ respectively, such that $C = A \cdot B$. The final beaver triples are obtained by sampling points from $A(\cdot)$, $B(\cdot)$ and $C(\cdot)$. For these triples to be random, it is crucial that the adversary does not learn $A(\cdot)$, $B(\cdot)$ and $C(\cdot)$. To be able to define $C(\cdot)$ of degree $2t$, $\mathcal{CS}$ must be of size at least $2t + 1$. However, this is a challenge in our setting since we have $n \leq 3t$. Following [BCV24], we pad $\mathcal{CS}$ to size $2t + 1$ with dummy triples. As a result, we need to ensure that $\mathcal{CS}$ contains triples from $t + 1$ honest parties. This way, the adversary learns at most t triples in the padded version of $\mathcal{CS}$, and does not learn $A(\cdot)$, $B(\cdot)$ and $C(\cdot)$.

We present an Agreement on Share Providers (ASP) protocol that agrees on a set of parties that have each secret shared a value, such that the set contains at least $t + 1$ honest parties. Similar to Π_{Collect} , the agreed-upon set will include $n - t$ parties and all parties synchronously connected to them. To this end, each party in the ASP protocol needs to identify all synchronously connected honest share providers within a bounded time. Compared to Π_{Collect} , the ASP protocol additionally needs to ensure that even when a corrupt share provider is identified, all parties either receive shares of this provider’s value, or agree to abort the protocol based on some evidence against the share provider.

To enable this, we upgrade our VSS protocol to one that additionally provides a *timely verification* property: once an honest dealer initiates VSS, all parties synchronously connected to it set a boolean flag to true within a bounded time B . Once this flag is set to true, the instance is guaranteed to terminate even if the dealer is corrupt. Each party locally forms a set of instances whose flags are set to true within time B . By agreeing on $n - t$ such sets using ACS (Agreement on Common Subset), parties agree on the output of ASP.

2 Related Work

It is well known that VSS [CGMA85, Fel87, CKLS02] and MPC with guaranteed output delivery [GMW87, BOGW19, BOKR94] can be achieved in synchrony and asynchrony if and only if up to a one-half and one-third of the parties are corrupt. Later works improve the efficiency of VSS [GRR98, BKP11, YLF⁺21, SS24] and MPC [RBO89, DN03, DI05, HNP05, Coh16]. Agreement on Common Subset was introduced in [BOCG93] for the purpose of asynchronous MPC and was later improved and used in other applications [MXC⁺16, DYX⁺22, DWZ23].

The work of [GAC⁺24] introduces the granular synchrony model, and presents deterministic protocols for byzantine agreement. In this work, we present protocols in granular synchrony for VSS, ACS and MPC with guaranteed output delivery. Our main motivation is to tolerate more than one-third corruptions without resorting to full synchrony. Protocols for MPC with abort (where not all parties may output) can tolerate a dishonest majority [Ors21] and is not the focus of this paper.

In [GPS19], the authors present a protocol for MPC in *weak synchrony*, a model that allows parties to go offline, so long as a majority of parties are honest and online in each round. In this sense, granular synchrony is a weaker assumption as it does not require a majority of the parties to communicate synchronously among themselves. In another sense, granular synchrony is a stronger assumption, as communication links are assumed to be static, while weak synchrony allows the set of online parties to change every round. Thus, the two models are formally incomparable.

Another line of work studies *network-agnostic* protocols, which are protocols that are secure in both synchronous and asynchronous networks, with parties being unaware of the network type [BLZL20, AC23, BCLZL23, BCV24]. These protocols assume that the network is either *fully* synchronous or *fully* asynchronous. As a consequence, these protocols cannot tolerate more than one-third corruptions when even a single honest party is partitioned.

Kamp and Nielsen [KN23] show that, assuming external common coins, any RBC protocol can be used to obtain an Agreement on Common Subset (ACS) protocol. Since this reduction holds in an asynchronous network when $t < n/2$, our RBC protocol immediately implies an ACS protocol in granular synchrony, again assuming the availability of external common coins. In this work, we construct ACS using RBC and VSS *without* assuming external common coins.

3 Preliminaries

3.1 Model

We use $\mathcal{P} = \{P_1, \dots, P_n\}$ to denote the set of parties in the MPC protocol. Up to $t < n/2$ parties may be controlled by the adversary $\mathcal{A}$, and these parties may deviate arbitrarily from the protocol. We assume the following. $\mathcal{A}$ is static, i.e., $\mathcal{A}$ decides which parties to corrupt before the protocol starts. $\mathcal{A}$ is computationally bounded, and we denote by λ the computational security parameter. We use Hon to denote the set of honest parties. We assume a public key infrastructure (PKI) setup: a pair of keys $(\text{pk}_i, \text{sk}_i)$ has been generated for each party $P_i \in \mathcal{P}$, where pk_i is public and sk_i is held only by P_i . We assume a collision resistant hash function H . Any digital signature and any hash is of size λ .

The PKI setup is used to establish a private and authenticated communication link between each pair of parties. We assume each link is bidirectional. In other words, the network is modeled by an *undirected* graph that defines whether each (bidirectional) link is synchronous or asynchronous. Messages sent on synchronous links are delivered within a finite, known time Δ . Messages sent on asynchronous links will be delivered eventually but the adversary may delay each such message for as long as it wishes. We also assume that the graph is static, i.e., it is not possible for a synchronous link to become asynchronous and vice versa during the execution of our MPC protocol. We use view_i to denote the transcript of messages seen by party P_i .

3.2 Notation

We introduce terminology to define how parties communicate. Throughout our protocols, parties use the PKI setup to encrypt and sign messages. We use $\langle v \rangle_i$ to denote a value v signed by party P_i . Next, we introduce notation to capture whether honest parties can communicate synchronously.

Definition 1 (Synchronously Connected Parties). *Two honest party P_i and P_j are **synchronously connected**, written as $P_i \leftrightarrow P_j$, if there exists a sequence of synchronous edges $(P_i, P_{\text{int}_1}), (P_{\text{int}_1}, P_{\text{int}_2}), \dots, (P_{\text{int}_k}, P_j)$ where each intermediate party P_{int_k} is honest.*

$\ell(P_i, P_j)$ denotes the length of the shortest synchronous path from P_i to P_j .

Definition 2 (Synchronous Extension of Party Set). For a set $A \subseteq \mathcal{P}$, $\text{Extended}(A) \subseteq \mathcal{P}$ denotes the union of set A and the set of honest parties synchronously connected to honest parties in A :

$$\text{Extended}(A) = A \cup \{P_j \in \text{Hon} \mid \exists P_i \in \text{Hon} \cap A \text{ such that } P_i \leftrightarrow P_j\}$$

The following notion captures a set of parties that includes all honest parties synchronously connected to the honest parties in the set.

Definition 3. A set of parties $A \subseteq \mathcal{P}$ is said to be **synchronously extended** if $A = \text{Extended}(A)$.

It is convenient to also describe parties that *cannot* communicate synchronously with each other.

Definition 4 (Partitioned Parties). We say two honest parties P_i and P_j are **partitioned**, denoted by $P_i \not\leftrightarrow P_j$, if they are not synchronously connected.

Definition 5 (Partitioned Party Sets). We say two sets of honest parties A and B are **partitioned**, denoted by $A \not\leftrightarrow B$, if $\forall a \in A, b \in B, a \not\leftrightarrow b$.

3.3 The Network Condition

We present below the granular synchrony network condition that is both necessary and sufficient to construct our protocols.

Condition 1. For any set Corr of $t < n/2$ parties, for all sets of honest parties $\text{Hon}_1, \text{Hon}_2 \subseteq \text{Hon}$, if $\text{Hon}_1 \not\leftrightarrow \text{Hon}_2$, then either $|\text{Hon}_1| < n - 2t$ or $|\text{Hon}_2| < n - 2t$.

[GAC⁺24] proves that Byzantine agreement is possible if and only if Condition 1 holds, even for randomized protocols.⁴ In §A, we show that Condition 1 is also necessary for RBC. This immediately implies that Condition 1 is necessary for ACS and MPC with guaranteed output delivery. Intuitively, this is because if Condition 1 does not hold, a set of t corrupt parties that includes the sender can delay all messages between the two partitioned sets of honest parties, while behaving as though the sender's input is 0 with one set, and is 1 with the other. This causes parties in the two sets to output different values, violating consistency. Since any VSS can be used to construct an RBC protocol, Condition 1 is required for VSS as well.

We assume that parties are aware of the maximum among the minimum length synchronous path between any two parties i.e. $\max_{P_i, P_j \in \mathcal{P}} \ell(P_i, P_j)$. We call this the *synchronous diameter*, denoted by D . $n - 1$ is a trivial upper bound. This is useful to measure the *round complexity* of our protocols. We measure both *synchronous* and *asynchronous* rounds. We assume lock-step synchrony, where each synchronous round takes Δ time. In asynchrony, a party proceeds to the next round if it has to send messages that are dependent on messages received during the previous round.

3.4 Primitives and Definitions

This section defines primitives used in our sub-protocols and MPC. The guarantees provided by these primitives differ in synchronous and asynchronous networks. Our definitions follow those used for asynchronous networks: this is the best that can be achieved since parties can be partitioned from each other in granular synchrony.

⁴ [GAC⁺24] stated the condition in a different but equivalent way: every set of honest parties of size $n - 2t$ is synchronously connected to at least $t + 1$ honest parties. In §A, we show that their condition is equivalent to our Condition 1. We use Condition 1 since it gives a clearer picture of how parties can be partitioned.

Multi Party Computation A protocol for MPC allows parties to jointly compute a function F without revealing their inputs. In this paper, we focus on MPC with guaranteed output delivery, i.e., all parties are guaranteed to learn the function output.

In asynchrony, it is not possible for all inputs from honest parties to be considered while computing F , as it is difficult to distinguish a corrupt party from one that is slow but honest. An MPC protocol is said to achieve L -output quality if L inputs are taken into account while computing F [BLZL20].

Definition 6 (Secure Multi Party Computation). *Suppose that each $P_i \in \mathcal{P}$ has an input x_i . An MPC protocol with guaranteed output delivery and L -output quality guarantees the following.*

- *Correctness* : $\exists CS \subseteq \mathcal{P}$ such that $|CS| \geq L$ and each party outputs a value $y = F(x_1, \dots, x_n)$, where $x_i = \perp$ (some default value) if $P_i \notin CS$.
- *Privacy* : The view of $\mathcal{A}$ is independent of x_i for each honest P_i .

It is well known that the best that asynchronous protocols can achieve is $(n - t)$ -output quality [BOKR94]. We present a protocol for MPC that always includes the inputs of a synchronously extended set of at least $n - t$ parties. In §E, we prove that this is the best that can be achieved in granular synchrony.

Verifiable Secret Sharing Verifiable Secret Sharing (VSS) is a common building block for MPC. A VSS protocol consists of two phases: *Sharing* and *Reconstruction*. The sharing phase allows a dealer D to provide each party with a share of a secret that can later be reconstructed in the reconstruction phase.

Definition 7 (Verifiable Secret Sharing). *Let D be a party with secret input $s \in \mathbb{F}$. Protocol Π_{VSS} is a **verifiable secret sharing protocol** if it provides the following properties, where each party outputs a share $s_i \in \mathbb{F}$.*

- *Correctness*: If D is honest, all parties output their share at the end of the sharing phase. If all honest parties start the reconstruction phase with their share, then all parties output s .
- *Privacy*: If D is honest, a computationally bounded adversary learns no information about s apart from the shares of corrupt parties.
- *Consistency*: If any honest party outputs a share in the sharing phase, then there exists some value $s^* \in \mathbb{F}$ such that all honest parties eventually output a share of s^* in the sharing phase. If all honest parties start the reconstruction phase with these shares, then all parties output s^* .

Our VSS protocol is based on Shamir secret sharing [Sha79], and ensures that each party receives a point from a dealer D such that all points held by honest parties lie on a common d -degree polynomial, where $d \geq t$. To verify that D has shared consistent points, we use a **polynomial commitment** scheme that generates a commitment to a polynomial of a fixed degree. This commitment can later be used to prove that a given evaluation point of the polynomial is correct, without revealing the polynomial.

Definition 8. A **polynomial commitment scheme** PC consists of the following algorithms:

- $PC.Setup(1^\kappa, d) \rightarrow pp$: Generates public parameters for the scheme to commit to a polynomial over $\mathbb{F}$ of degree d . These parameters are input to all other functions provided by PC , and are omitted for clarity.

- $\text{PC.Commit}(p(\cdot), n) \rightarrow (\text{com}, \mathbf{w})$: Given a polynomial $p(\cdot)$ and the number of evaluations n , outputs a witness $\mathbf{w}$ and a commitment com .
- $\text{PC.Open}(\mathbf{w}, p(\cdot), i) \rightarrow (p(i), \pi)$: Evaluates a polynomial at a given index i and generates a proof π that can be used to check that the evaluation is correct.
- $\text{PC.Verify}(\text{com}, d, i, u, \pi) \rightarrow \{0, 1\}$: Given a polynomial commitment com , outputs 1 if com is a commitment to a polynomial $p(\cdot)$ of degree at most d , and if $u = p(i)$; it outputs 0 otherwise.

We require the polynomial commitment scheme to satisfy the standard properties of *correctness*, *hiding*, and *binding*. A polynomial commitment scheme is *hiding* if, except for negligible probability, an adversary that is provided with the polynomial commitment and up to d evaluations of the polynomial *cannot* learn an evaluation of the polynomial that is not provided to it. It is *binding* if, except for negligible probability, the adversary cannot generate a valid proof for a point not on the polynomial. It is *correct* if every proof of evaluation that is generated is successfully verified. [KZG10] provides a formal definition.

We additionally require the polynomial commitment scheme to be *homomorphic*: it is possible to compute the commitment com_r to any linear combination $r(\cdot)$ of two polynomials $p(\cdot)$ and $q(\cdot)$, from their commitments com_p and com_q . Similarly, a proof of evaluation of $r(\cdot)$ at an index can be generated using a linear combination of evaluation proofs of $p(\cdot)$ and $q(\cdot)$ at that index. This property is important for evaluating linear gates in MPC. Polynomial commitment schemes that satisfy these required properties are known assuming that the discrete logarithm problem is hard [Fel87, Ped91].

Agreement. We present definitions of primitives that help us agree on a set of parties that have verifiably secret shared their inputs.

In an Externally Valid Byzantine Agreement protocol (EVBA), each party's input and the common output satisfy an external predicate that checks if a certain condition holds. In our case, the predicate depends on a party's *view* during the protocol.

Definition 9 (View). *The transcript of messages that a party receives.*

Note that when a party starts executing a sub-protocol, its view consists of messages that it receives before the start of the protocol. A party's view can be used as input to a predicate. For instance, to check that a designated dealer has shared a secret value, such a predicate may check that a party's view contains their shares of the dealer's secret.

Definition 10 (View-based Predicates). *A view-based predicate Q is a predicate that takes a party's view and a value as input to output a boolean value. If $Q(\text{view}_i, v)$ holds true at time T for any honest party P_i , then -*

1. $Q(\text{view}_i, v)$ holds true at any time $T' \geq T$.
2. Each honest P_j eventually has a view view_j for which $Q(\text{view}_j, v)$ holds true.

Definition 11 (Externally Valid (Binary) Byzantine Agreement). *Suppose that each party P_i holds an input bit b_i . Protocol Π_{EVBA} is an **externally valid (binary) byzantine agreement protocol with view-based predicate Q** that outputs a bit b if the following properties hold.*

- *Agreement:* If any honest party outputs b , then all honest parties eventually output b .
- *External Validity:* If any honest P_i outputs b , then $Q(\text{view}_i, b)$ eventually holds true.
- *Termination:* If each honest P_i participates with b_i such that $Q(\text{view}_i, b_i)$ holds, then all parties eventually output.

An Agreement on Common Subset (ACS) protocol outputs a common set of inputs provided by at least $n - t$ parties that satisfy a view-based predicate.

Definition 12. Suppose that each $P_i \in \mathcal{P}$ has input v_i . Protocol Π_{ACS} in an **Agreement on a Common Subset protocol with view-based predicate Q** if the following hold.

- *Validity:* If an honest party P_i outputs a set $\mathbf{v}$, then $|\mathbf{v}| \geq n - t$ and $\mathbf{v}$ contains the inputs of at least $n - 2t$ honest parties, and $Q(\text{view}_i, v)$ holds for each $v \in \mathbf{v}$.
- *Agreement:* If an honest party outputs $\mathbf{v}$, then every honest party outputs $\mathbf{v}$.
- *Termination:* If $n - t$ honest parties P_i participate with an input v_i such that $Q(\text{view}_i, v_i)$ holds, then all honest parties output $\mathbf{v}$.

Underlying all our protocols is reliable broadcast (RBC), a primitive that lets all parties agree on a message sent by a party S called the sender. If S does not send anything, the parties need not output.

Definition 13 (Reliable Broadcast). Let S be a party with input message m . Π_{RBC} is a **reliable broadcast protocol** if it provides the following properties.

- *Validity:* If S is honest, each honest party outputs m .
- *Consistency:* If some honest party outputs a message m' (which may or may not be m), each honest party outputs m' .

4 Collect

In this section, we present Π_{Collect} – a key building block underlying our approach. Informally, each party participates in Π_{Collect} with an input value and outputs a *collection* of signed inputs. The collection must include signatures from a *synchronously extended* set of $n - t$ parties (and possibly more). Π_{Collect} guarantees that each input satisfies a view-based predicate Q_{in} , and that the output collection satisfies a static predicate Q_{out} ⁵. Before we describe Π_{Collect} , we define how synchronously connected parties participate in Π_{Collect} .

Protocol $\Pi_{\text{Collect}}(Q_{\text{in}}, Q_{\text{out}})$

- On obtaining input v_i , send $\langle v_i \rangle_i$ to all parties in $\mathcal{P}$.
- Initialize $\mathcal{I}_i$, the set of signed inputs of parties connected to P_i , to $\emptyset$.
- Wait for time $2D\Delta$. During this time, on receiving $\langle v \rangle_j$ from any party for the first time such that $Q_{\text{in}}(\text{view}_i, v)$ holds, send $\langle v \rangle_j$ to all parties and set $\mathcal{I}_i = \mathcal{I}_i \cup \langle v \rangle_j$.
- Send $\langle \mathcal{I}_i \rangle_i$ to all parties in $\mathcal{P}$. On receiving $\langle \mathcal{I} \rangle_j$, if $Q_{\text{in}}(\text{view}_i, v)$ holds for each $\langle v \rangle_k \in \mathcal{I}_j$, set $\mathbf{I}_i = \mathbf{I}_i \cup \langle \mathcal{I} \rangle_j$.
- Upon having $\mathbf{I}$ (which may be $\mathbf{I}_i$) that satisfies the following:
 1. $|\mathbf{I}| = n - t$,
 2. $Q_{\text{in}}(\text{view}_i, v)$ holds for each $\langle v \rangle_j \in \mathbf{I}$, and
 3. $Q_{\text{out}}(\mathbf{I})$ holds true,
 send $\mathbf{I}$ to all parties, output $\mathcal{C}_i = \mathbf{I}$, and terminate.

Fig. 1: Code for party P_i to output a set $\mathcal{C}_i$ that contains inputs from a synchronously extended set of at least $n - t$ parties. Each P_i participates with input v_i .

⁵ For our protocols, it suffices for Q_{out} to be a static (and not view-based) predicate.

Requirements from synchronously connected parties. How we exploit synchronous communication links is key to designing protocols in granular synchrony. At a high level, we require that if any party performs a step at a certain time, parties synchronously connected to it perform the same step within a bounded time. Below, we define the notions capturing that synchronously connected parties must participate in Π_{Collect} within bounded time of each other, and that their views for the purpose of validation must be loosely synchronized within bounded time.

Definition 14 (Timely Participation). *A protocol Π has **timely participation** if, once an honest P_i participates with an input at time T , every honest P_j synchronously connected to it participates with an input within time $T + D\Delta$.*

Any view-based predicate externally provided to Π_{Collect} must additionally satisfy what we call *timely validation*.

Definition 15 (Timely Validation). *A view-based predicate Q is guaranteed **timely validation** if, once $Q(\text{view}_i, v)$ holds true, where view_i is P_i 's view at time T , then $Q(\text{view}_j, v)$ holds true for every honest P_j synchronously connected to P_i , where view_j is P_j 's view after time $T + \ell(P_i, P_j)\Delta$.*

These notions enforce how protocols must invoke Π_{Collect} , and provide a means for composition. Looking ahead, all protocols that invoke Π_{Collect} will ensure timely participation for Π_{Collect} and timely validation for Q_{in} . If timely participation holds and Q_{in} has timely validation, then Π_{Collect} satisfies the following properties:

- **Correctness:** If a party outputs a collection $\mathcal{C}$, then 1) $\mathcal{C}$ is a synchronously extended set of size at least $n - t$ 2) each input in $\mathcal{C}$ is validated by Q_{in} 3) $\mathcal{C}$ is validated by Q_{out} .
- **Termination:** If each honest party participates with a valid input, and if Q_{out} is defined such that a collection of valid inputs is a valid collection, then all honest parties eventually output a collection.
- **Timely Termination:** If an honest party outputs a collection at time T , then all honest parties synchronously connected to it output a collection within time $T + D\Delta$.
- **Honest Majority:** A collection contains at least $t + 1$ honest parties' inputs.
- **Honest Intersection:** The intersection of two collections contains at least one honest party's input.

Description of Π_{Collect} . We present Π_{Collect} in Figure 1. Π_{Collect} hinges on the observation that if parties relay their inputs to one another, then given timely participation, a party will receive inputs from all parties synchronously connected to it if it waits for $2D\Delta$ time. Each P_i collects externally valid inputs into an intermediary set $\mathcal{I}_i$. Given timely validation, all inputs from honest parties synchronously connected to P_i are included in $\mathcal{I}_i$.

Parties then send these intermediary sets to one another. $n - t$ of these intermediary sets form what we call a “packed” collection (denoted $\mathbf{I}$), and it contains the inputs of the parties that sent these sets, along with parties synchronously connected to them. A party's input may be present in multiple $\mathcal{I}$ sets in $\mathbf{I}$, and the final output collection is obtained by taking the union of these sets, and checking that it satisfies the predicate Q_{out} .

Lemma 1 (Correctness). *Given timely participation for Π_{Collect} and timely validation for Q_{in} , if an honest P_i outputs $\mathcal{C}_i$, then $\mathcal{C}_i$ contains a set of signed inputs from a synchronously extended set of parties of size at least $n - t$ where 1) each input in $\mathcal{C}_i$ is of the form $\langle v \rangle_j$ where $Q_{\text{in}}(\text{view}_i, v)$ holds and 2) $Q_{\text{out}}(\mathcal{C}_i)$ holds.*

Proof. Suppose that P_i outputs $\mathcal{C}_i$. From inspection of Figure 1, $\mathcal{C}_i = \cup \mathbf{I}$, where $\mathbf{I}$ is a set of $n - t$ $\mathcal{I}$ sets. Define A to be the set of parties P_j for which $\langle \mathcal{I} \rangle_j \in \cup \mathbf{I}$. As discussed, $\mathbf{I}$ contains inputs from all honest parties in $\text{Extended}(A)$, a synchronously extended set. The proof follows since P_i checks that $|\mathbf{I}| \geq n - t$, each input in $\mathcal{C}$ is of the form $\langle v \rangle_j$ where $Q_{\text{in}}(\text{view}_i, v)$ holds, and that $Q_{\text{out}}(\cup \mathbf{I})$ holds. $\square$

Protocols that invoke Π_{Collect} are responsible for defining the predicate Q_{in} that dictates what inputs a party can participate in Π_{Collect} with, and the predicate Q_{out} that constrains the types of collection that can be output. Π_{Collect} terminates when these predicates are defined such that a valid set of inputs is bound to form a valid collection.

Lemma 2 (Termination). *Each honest party P_i eventually outputs some collection $\mathcal{C}_i$ if*

1. *Each honest party participates in Π_{Collect} with an input v for which $Q_{\text{in}}(v)$ is true*
2. *Predicates Q_{in} and Q_{out} are defined such that for any $\mathbf{I}$, if $Q_{\text{in}}(\text{view}_i, v)$ holds for each $\langle v \rangle_j \in \cup \mathbf{I}$, then $Q_{\text{out}}(\cup \mathbf{I})$ also holds true.*

Proof. Each honest P_j waits for time $2D\Delta$, and sends its intermediary set $\mathcal{I}_j$ to all other parties. (Note that even in a purely asynchronous network when $t < n/3$, $\mathcal{I}_j$ is non-empty as P_j includes its own input.) Thus, P_i receives at least $n - t$ intermediary $\mathcal{I}_j$ sets and adds them to $\mathbf{I}_i$. From inspection of Figure 1, for each $\langle v \rangle_j \in \cup \mathbf{I}_i$, $Q_{\text{in}}(\text{view}_i, v)$ holds. Thus, $Q_{\text{out}}(\cup \mathbf{I})$ also holds, and P_i outputs $\mathcal{C}_i = \cup \mathbf{I}_i$. $\square$

Π_{Collect} also offers *timely termination* as a useful property. If any party outputs a collection at time T , then all parties connected to it output a collection within time $T + D\Delta$. This is particularly useful when we call multiple instances of Π_{Collect} one after the other – the timely termination of one instance of Π_{Collect} ensures timely participation for the *next* instance of Π_{Collect} .

Lemma 3 (Timely Termination). *Suppose that Q_{in} satisfies timely validation. If P_i outputs $\mathcal{C}_i$ at time T , then each P_j eventually outputs $\mathcal{C}_j$, and it happens by time $T + D\Delta$ if $P_i \leftrightarrow P_j$.*

Proof. Let $\mathcal{C}_i = \cup \mathbf{I}$. P_i forwards $\mathbf{I}$ to all parties. Each party, on seeing a packed collection $\mathbf{I}$ of size $n - t$ containing inputs that are externally valid, relays this to all other parties. If $P_i \leftrightarrow P_j$, P_j receives some packed collection $\mathbf{I}'$ within time $D\Delta$. From the timely validation of Q_{in} , P_j checks that $\mathbf{I}'$ is “correct”, and outputs a collection. Otherwise, P_j eventually receives $\mathbf{I}$ and outputs $\mathcal{C}_j$. $\square$

The collections output Π_{Collect} have two useful properties: *honest majority* and *honest intersection*. Our RBC and VSS protocols rely on these properties.

Lemma 4 (Honest Majority). *Suppose that P_i outputs $\mathcal{C}_i$. $\mathcal{C}_i$ contains at least $t+1$ honest parties’ signed inputs.*

Proof. By correctness of Π_{Collect} , $\mathcal{C}_i$ contains inputs from a synchronously extended set of parties of size at least $n - t$. Let Hon_{in} denote those honest parties whose inputs are included in this set, and Hon_{out} denote those honest parties whose inputs are *not* included in this set. Since there are at most t corruptions, $|\text{Hon}_{\text{in}}| \geq n - 2t$. Note that party sets Hon_{in} and Hon_{out} must be *partitioned*. Since $|\text{Hon}_{\text{in}}| \geq n - 2t$, Condition 1 enforces that $|\text{Hon}_{\text{out}}| < n - 2t$. Thus:

$$|\text{Hon}_{\text{in}}| = |\text{Hon} \setminus \text{Hon}_{\text{out}}| > (n - t) - (n - 2t) = t. \quad \square$$

Lemma 5 (Honest Intersection). *Suppose that P_i and P_j output $\mathcal{C}_i$ and $\mathcal{C}_j$ respectively. Then, $\mathcal{C}_i \cap \mathcal{C}_j$ contains at least one honest party’s signed input.*

Proof. By correctness of Π_{Collect} , $\mathcal{C}_i$ contains inputs from a synchronously extended set of at least $n - t$ parties. Let Hon_i denote those honest parties whose inputs are included in this set. Let us similarly define Hon_j for $\mathcal{C}_j$. Since there are at most t corruptions, $|\text{Hon}_i| \geq n - 2t$; similarly, $|\text{Hon}_j| \geq n - 2t$. For the sake of deriving a contradiction, suppose that $\text{Hon}_i \cap \text{Hon}_j$ contains no honest party's input. As Hon_i and Hon_j are synchronously extended sets, this means that Hon_i and Hon_j are *partitioned* from each other. But $|\text{Hon}_i| \geq n - 2t$ and $|\text{Hon}_j| \geq n - 2t$. This contradicts Condition 1. $\square$

Cost: Π_{Collect} incurs a communication cost of $O(n^4|v| + n^4\lambda)$ bits, where $|v|$ denotes the maximum size of any value in the domain of Q_{in} . This is because each party relays a packed collection of size $O(n^2|v| + n^2\lambda)$ bits to every other party. Π_{Collect} incurs $O(D)$ synchronous rounds, and $O(1)$ asynchronous rounds.

5 Reliable Broadcast

In this section, we present our reliable broadcast protocol Π_{RBC} . Π_{RBC} proceeds by first having S send $\langle m \rangle_S$ to all parties. Since a corrupt S may send different messages to different parties, to agree on S 's message, parties participate in an instance of Π_{Collect} with the (signed) message received from S as input. Specifically, parties invoke Π_{Collect} with Q_{in} that only accepts messages signed by S , and Q_{out} that outputs a collection of signatures on a *common* value.

Since an honest S sends the same $\langle m \rangle_S$ to all parties, Q_{in} implies that Q_{out} holds, and Π_{Collect} terminates. When S is corrupt, if Π_{Collect} terminates, then the honest intersection of Π_{Collect} ensures that parties output consistent values.

We present Π_{RBC} in Figure 2. Before we prove the properties of Π_{RBC} , we argue that we invoke Π_{Collect} in the correct way. Timely participation is guaranteed in Π_{Collect} since each party forwards the first message from S that it sees. The timely validation of Q_{in} holds because Q_{in} does not depend on individual views.

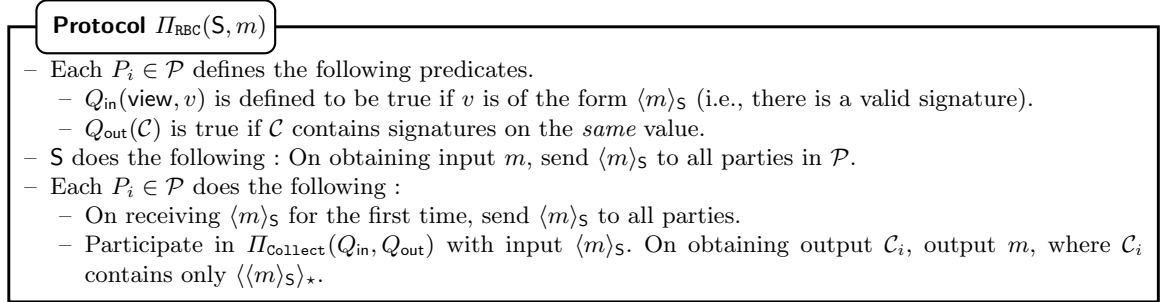


Fig. 2: Reliable broadcast protocol for sender S

Lemma 6 (Validity). *When S is honest, all parties eventually output m .*

Proof. Since S is honest, S sends $\langle m \rangle_S$ to all parties, and each honest party P_i participates in Π_{Collect} with input $\langle m \rangle_S$ to obtain output $\mathcal{C}_i$. Since S sends only $\langle m \rangle_S$ to all parties, by the security of the signature scheme, only $Q(\star, \langle m \rangle_S)$ is valid, implying that $Q_{\text{out}}(\mathcal{C}_i)$ holds true. From the termination of Π_{Collect} , P_i outputs $\mathcal{C}_i$ that contains only $\langle \langle m \rangle_S \rangle_{\star}$, and outputs m . $\square$

Similar to Π_{Collect} , Π_{RBC} also offers timely termination. This will be useful to ensure timely participation in an instance of Π_{Collect} invoked after Π_{RBC} .

Lemma 7 (Timely Termination). *If any party P_i outputs a value m_i at time T , then all honest parties P_j eventually output a value m_j , where this is done within time $T + D\Delta$ if $P_i \leftrightarrow P_j$.*

Proof. Immediate from the timely termination of Π_{Collect} . $\square$

Lemma 8 (Consistency). *If any party outputs m' , then all parties output m' .*

Proof. Consider party P_i that outputs $m_i = m'$, say at time T . This means that P_i has output $\mathcal{C}_i$ such that $\mathcal{C}_i$ contains only $\langle m' \rangle_S$. From Lemma 7, all parties output some value. For contradiction, suppose that some party P_j outputs $m_j \neq m'$, i.e., P_j outputs a collection $\mathcal{C}_j$ that contains only $\langle m'' \rangle_S$, where $m'' \neq m'$. From Lemma 5, $\mathcal{C}_i \cap \mathcal{C}_j$ contains at least one honest party's (say P_h) signed input. This means that P_h participated in Π_{Collect} with inputs $\langle m' \rangle_S$ and $\langle m'' \rangle_S$, which is a contradiction. $\square$

Cost: The total communication required in Π_{RBC} is $O(n^4|m| + n^4\lambda)$. This is since the communication cost of Π_{Collect} is $O(n^4|v| + n^4\lambda)$, where v is the input passed to Π_{Collect} , and $|v| = |m| + \lambda$ in this case. Due to the round complexity of Π_{Collect} , Π_{RBC} incurs $O(D)$ synchronous rounds and $O(1)$ asynchronous rounds.

Improving Efficiency for Long Messages We reduce the communication cost of Π_{RBC} to $O(n^2|m| + n^4\lambda)$ bits. Looking ahead, we will use this to reduce the communication cost of our VSS protocol. Recall that the consistency of Π_{RBC} is based on the honest intersection of collections of *messages*. We can instead make parties participate with the signed *hash* of a message as input in Π_{Collect} . If the hash function is collision-resistant, consistency through honest intersection still holds. Since this hash is of size λ which may be much smaller than m for long messages, this saves communication.

Note that if some party outputs a collection of some hash value h , then all parties will output a collection of h . However, for termination, parties additionally need to know what *message* to output. To facilitate this, each party relays the (unique) message m signed by S such that $H(m) = h$. We present Π_{RBC} for long messages and formally prove its correctness in Appendix B.

6 Verifiable Secret Sharing

In this section, we describe our verifiable secret sharing protocol Π_{VSS} . A designated party D called the dealer inputs a secret s , and each party P_i outputs a (distinct) point on a polynomial $p(\cdot)$ whose constant term is s . We further require Π_{VSS} to be *complete*, i.e., either *every* party outputs a point or *no one* outputs. This is important for designing our MPC protocol.

Our protocol Π_{VSS} is adapted from the protocol of [DXT⁺23]. D samples a t -degree polynomial $p(\cdot)$ with $p(0) = s$, and generates a polynomial commitment com to p . D also sends to each party P_i its share $s_i = p(i)$. Note that a corrupt D may send shares to parties that do not lie on the same polynomial. To ensure that this is not the case, on receiving its share, P_i uses com to check that s_i lies on the committed polynomial. The protocol needs to check that enough parties have received points that lie on the polynomial, so that they are able to reconstruct it later.

To do so, each P_i participates in Π_{Collect} either with input $\langle \text{OK} \rangle_i$, indicating that it has locally checked that it has received a correct point, or with input $\langle \text{NOK} \rangle_i$, indicating that it has received an incorrect point. Any collection of OK and NOK messages is considered to be valid, and so, Π_{Collect} is guaranteed to terminate. D then broadcasts a collection $\mathcal{C}_D$ along with the points of each party P_i

for which $\langle \text{OK} \rangle_i$ is *not* included in $\mathcal{C}_D$: this includes parties for which $\langle \text{NOK} \rangle_i$ is in $\mathcal{C}_D$, and parties whose inputs are not included $\mathcal{C}_D$. In this way, *all* parties receive their shares, and Π_{VSS} is complete.

To ensure that privacy holds, D uses a polynomial of degree $n - t - 1$ to share s , instead of using a t -degree polynomial. We present Π_{VSS} in Figure 3. Lines highlighted in blue are extra steps used required to agree on the set of input providers for MPC. Reader can ignore these steps for now; §8.1 provides the intuition behind these steps.

Protocol $\Pi_{\text{VSS}}^+(D, s)$

- Each P_i defines the following predicates.
 - $Q_{\text{in}}(\text{view}, v)$ is true if $v \in \{\text{OK}, \text{NOK}\}$.
 - $Q_{\text{out}}(\mathcal{C})$ is always true.
- D does the following.
 - Choose a $n - t - 1$ -degree polynomial $p(\cdot)$ uniformly at random such that $p(0) = s$.
 - Run $\text{PC.Commit}(p(\cdot), n) \rightarrow (\text{com}, w)$. Initialize set **shares** to $\emptyset$. For each $i \in [n]$, do the following.
 - Run $\text{PC.Open}(w, p(\cdot), i) \rightarrow (p(i), \pi_i)$.
 - Add $\text{Enc}_{pk_i}(i, p(i), \pi_i)$ to **shares**.
 - Participate in protocol $\Pi_{\text{RBC}}^{+(1)}(D, (\text{shares}, \text{com}))$
- Each P_i does the following :
 - Set int_i to 1 on setting int_i to 1 in protocol $\Pi_{\text{RBC}}^{+(1)}$.
 - Set flag_i to 1 on setting corrupt_i to 1 in protocol $\Pi_{\text{RBC}}^{+(1)}$.
 - On receiving an output from $\Pi_{\text{RBC}}^{+(1)}$, check that it is of the form $(\{c_1, \dots, c_n\}, \text{com})$. If it is not, set flag_i to 1.
 - Let $(i, p_i, \pi_i) = \text{Dec}_{sk_i}(c_i)$. If $\text{PC.Verify}(\text{com}, n - t - 1, i, p_i, \pi_i) \rightarrow 1$, set $v_i = \text{OK}$. Else, set $v_i = \text{NOK}$. Participate in $\Pi_{\text{Collect}}(Q_{\text{in}}, Q_{\text{out}})$ with input v_i .
- D does the following : Participate in $\Pi_{\text{RBC}}^{+(2)}(D, (\mathcal{C}_D, \{(i, p(i), \pi_i)\}_{\langle \text{OK} \rangle_i \notin \mathcal{C}_D}))$.
- Each P_i does the following :
 - Let T be the time at which $\mathcal{C}_i$ is obtained as the output of Π_{Collect} . If int_i is not set to 1 in $\Pi_{\text{RBC}}^{+(2)}$ within time $T + 2D\Delta$, set flag_i to 1.
 - Set flag_i to 1 on setting corrupt_i to 1 in protocol $\Pi_{\text{RBC}}^{+(2)}$.
 - On receiving output from $\Pi_{\text{RBC}}^{+(2)}$, set flag_i to 1 if any of the following do not hold.
 - Output is of the form $(\mathcal{C}_D, \{(j, p_j, \pi_j)\}_{\langle \text{OK} \rangle_j \notin \mathcal{C}_D})$ where each value in $\mathcal{C}_D$ is of the form $\langle \text{OK} \rangle_\star$ or $\langle \text{NOK} \rangle_\star$.
 - For each P_j for which $\langle \text{OK} \rangle_j \notin \mathcal{C}_D$, $\text{Verify}(\text{com}, n - t - 1, j, p_j, \pi_j) \rightarrow 1$.
 - If the above hold, output s_i as follows.
 - If $\langle \text{OK} \rangle_i \in \mathcal{C}_D$, output $s_i = p_i$ obtained as part of $\Pi_{\text{RBC}}^{+(1)}$.
 - Else, output $s_i = p_i$ obtained as part of $\Pi_{\text{RBC}}^{+(2)}$.

Fig. 3: VSS protocol for dealer D with input s . Lines highlighted in blue are additional steps used for timely verification.

Note that Π_{VSS} must ensure timely participation in the underlying instance of Π_{Collect} . Thus, D sends to each party an encrypted set of *all* parties shares, in contrast to sending only an individual share. In Figure 3, D broadcasts this set, and the timely termination of Π_{RBC} ensures timely participation in Π_{Collect} . We highlight that this is why we choose *not* to use a bivariate-based VSS [BOGW19], as relaying shares on a bivariate blows up communication by a factor of $O(n)$. Note further that we also improve efficiency by using Π_{RBC} for *long* messages.

Q_{in} in Figure 3 has timely validation since its output depends only on the value being validated, and is independent of each party's view.

Lemma 9 (Correctness). *If D is honest, each honest P_i outputs $s_i = p(i)$, where $p(\cdot)$ is a random $n - t - 1$ -degree polynomial with $p(0) = s$.*

Proof. From the validity of $\Pi_{\text{RBC}}^{(1)}$, each P_i receives $(\text{shares}, \text{com})$ as the output of $\Pi_{\text{RBC}}^{(1)}$ and checks that it is of the correct form. P_i also checks that $\text{PC.Verify}(\text{com}, i, p_i, \pi_i) \rightarrow 1$, where p_i and π_i are obtained by decrypting the i th ciphertext in shares . P_i thus participates in Π_{Collect} with input $\langle \text{OK} \rangle_i$. Note that since $Q_{\text{out}}(\mathcal{C})$ is true for any collection $\mathcal{C}$, from the termination of Π_{Collect} , D eventually outputs $\mathcal{C}_D$.

D broadcasts $\mathcal{C}_D$, along with the points of parties whose inputs are not included in $\mathcal{C}_D$, through $\Pi_{\text{RBC}}^{(2)}$. From the validity of $\Pi_{\text{RBC}}^{(2)}$, each P_i eventually receives $(\mathcal{C}_D, \{(j, p_j, \pi_j)\}_{\langle \text{OK} \rangle_j \notin \mathcal{C}_D})$ as the output of $\Pi_{\text{RBC}}^{(2)}$. P_i verifies that $\mathcal{C}_D$ is correct, and that $\text{PC.Verify}(\text{com}, j, p_j, \pi_j)$ holds for all broadcasted points. Thus, P_i outputs $s_i = p(i)$ where $p(i)$ is received through either instance of Π_{RBC} . $\square$

Lemma 10 (Privacy). *For an honest D with input s , the view of a computationally bounded $\mathcal{A}$ should be indistinguishable from its view when D 's input is s' , for any $s' \neq s$, except with negligible probability.*

Proof. We prove that $\mathcal{A}$ learns fewer than $n - t$ points on the polynomial $p(\cdot)$ of degree $n - t$. Let Hon_D be the set of honest parties whose inputs are included in $\mathcal{C}_D$; each of these inputs is of the form $\langle \text{OK} \rangle_*$. From Lemma 4, $|\text{Hon}_D| > t$. Apart from the points held by corrupt parties, $\mathcal{A}$ learns the points of honest parties in $\overline{\text{Hon}}_D = \text{Hon} \setminus \text{Hon}_D$. Since $|\text{Hon}_D| > t$, $|\overline{\text{Hon}}_D| < n - 2t$. Thus, $\mathcal{A}$ learns fewer than $n - 2t + t = n - t$ points on $p(\cdot)$. $\square$

Lemma 11 (Consistency). *If any honest P_i outputs a value for s_i at time T , then except with negligible probability, each honest P_j eventually outputs $s_j = p'(j)$, where $p'(\cdot)$ is a polynomial of degree $n - t - 1$.*

Proof. Note that as a result of the call to $\Pi_{\text{RBC}}^{(1)}$, P_i outputs a set of encrypted points and a commitment com . P_i also outputs $(\mathcal{C}_D, \{(j, p_j, \pi_j)\}_{\langle \text{OK} \rangle_j \notin \mathcal{C}_D})$ from $\Pi_{\text{RBC}}^{(2)}$. Each honest party P_j whose inputs are included in $\mathcal{C}_D$ locally checks that $\text{Verify}(\text{com}, j, p(j), \pi_j) \rightarrow 1$ holds. P_i further checks that $\text{Verify}(\text{com}, j, p_j, \pi_j) \rightarrow 1$ holds for all $P_j \in \mathcal{P}$ for which $\langle \text{OK} \rangle_j \notin \mathcal{C}_D$. Thus, P_i confirms that all honest parties receive points that lie on some polynomial $p'(\cdot)$ of degree $n - t - 1$. The rest of the proof follows from the consistency of Π_{RBC} . $\square$

Cost: Π_{VSS} incurs a communication of $O(n^4|v| + n^4\lambda)$ bits. This cost is due to calls to Π_{RBC} , each incurring a cost of $O(n^4\lambda)$ bits. Π_{VSS} incurs $O(D)$ synchronous rounds and $O(1)$ asynchronous rounds.

Improving Efficiency : For a batch of L secrets, the cost of invoking Π_{RBC} (for long messages) is $O(L \cdot (n^3|v| + n^4\lambda))$ bits. Here, the cost of invoking Π_{Collect} , being $O(n^4\lambda \cdot L)$ bits, dominates instead. This cost can be reduced to $O(n^3|v| \cdot L + n^4\lambda)$ bits by invoking a *single* instance of Π_{Collect} and having D broadcast a *single* collection for all L secrets. Parties participate in Π_{Collect} with input OK only if *all* L points are verified using the respective polynomial commitments. We present our batched VSS in §C.

Reconstructing a secret: The reconstruction protocol is straightforward and is presented in §C. To reconstruct a secret, each party sends its share s_i to all parties, along with the evaluation proof π_i . Each party, on receiving a share and its proof, can use `com` to verify that the share lies on the committed polynomial. On receiving $n - t$ correct shares, the committed polynomial can be reconstructed via interpolation. Since there are at least $n - t$ honest parties, all parties eventually reconstruct the polynomial.

Notation for shares of a secret: Note that a party’s share, the proof of evaluation, and the polynomial commitment are used for reconstruction. Thus, we use $\llbracket s \rrbracket_i$ to denote the tuple (s_i, π_i, com) , which is P_i ’s share. When understood, for readability, we drop the subscript and simply use $\llbracket s \rrbracket$.

On the linearity of Π_{VSS} : From the homomorphic property of the polynomial commitment, given $\llbracket a \rrbracket$ and $\llbracket b \rrbracket$, a party can locally compute its share of any linear combination c of a and b .

7 Randomized Agreement

In this section, we present our ACS protocol Π_{ACS} . We first present an EVBA protocol Π_{EVBA} that is used to agree on an externally valid output. We then use a standard construction that uses parallel instances of Π_{EVBA} and Π_{RBC} to construct Π_{ACS} . Looking ahead, a main challenge in MPC is in deciding on which parties’ inputs to use as part of the computation. Π_{ACS} solves this exact problem.

7.1 Externally Valid Byzantine Agreement (EVBA)

Each party participates in an externally valid byzantine agreement protocol with an input bit to agree on an output bit that satisfies an externally provided view-based predicate Q . We present a way to construct Π_{EVBA} from Π_{RBC} in a *black box* manner, assuming that $n > 2t$ holds. We further assume that parties have access to a global strong common coin; we discuss how to eliminate this assumption while constructing Π_{ACS} in Section 7.2. We remark that our protocol is similar to the BA protocol given in [Tou84]. In a way, we extend their result from $n > 3t$ in asynchrony to $n > 2t$ in granular synchrony under Condition 1.

Our protocol follows the standard template of asynchronous BA protocols that use a strong common coin. It proceeds in iterations, where, in each iteration, parties participate with an “externally valid” input. For the first iteration, external validity is provided by the predicate Q . For each subsequent iteration, external validity is defined using the previous iteration. During any iteration, if only a single input bit is externally valid, then parties reach agreement. Eventually, all parties (even corrupt ones) start an iteration with a common bit as input, ensuring termination.

We present Π_{EVBA} in Figure 4. Each iteration consists of two phases, with P_i participating with input u_i for the first phase and v_i for the second. In each phase p , parties broadcast their inputs using reliable broadcasts (RBC), and each P_i keeps track of the count of values that have been heard from the broadcast of other parties in each phase (using U_i and V_i). A party participates in phase 2 with input b if it receives b from the broadcast of $n - t$ parties in phase 1. Otherwise, it participates in phase 2 with input $\perp$.

On receiving the broadcast of $n - t$ parties during phase 2 of an iteration, each party either decides its output, or it decide its input for phase 1 of the next iteration. If a party receives a bit b from *all* broadcasts, then it outputs b . On the other extreme, if a party receives $\perp$ from *all* broadcasts, then it flips a coin and participates in the next iteration with the output of the coin. Otherwise, it receives both b and $\perp$, and participates in the next iteration with b .

If a party receives value v from the broadcast of $n - t$ parties in either phase, then it cannot receive value $v' \neq v$ from $n - t$ parties in the same phase. This is captured in the following lemma.

Protocol $\Pi_{\text{EVBA}}(Q)$

- On obtaining input b_i , set u_i to be b_i .
- For $\text{iter} = 1, 2, \dots$ do the following.^a
 - Set U_i to be a dictionary mapping values in $\{0, 1\}$ to a count (initially 0).
 - Participate in $\Pi_{\text{RBC}}^{(i,1)}(P_i, u_i)$.
 - If $u \in \{0, 1\}$ is output in $\Pi_{\text{RBC}}^{(j,1)}$, set $U_i[u] = U_i[u] + 1$ if the following hold.
 - * If $\text{iter} = 1$, then $Q(\text{view}_i, u)$ holds.
 - * Else, either of the following hold.
 - $u = v_i$.
 - $u = \text{Coin}(\text{iter} - 1)$ and $V_i[\perp] \geq n - t$ (in $\text{iter} - 1$).
 - Once $U_i[0] + U_i[1] = n - t$, set v_i to be
 - * b , if $U_i[b] = n - t$.
 - * $\perp$ otherwise.
 - Set V_i to be a dictionary mapping values in $\{0, 1, \perp\}$ to a count (initially 0).
 - Participate in $\Pi_{\text{RBC}}^{(i,2)}(P_i, v_i)$.
 - If $v \in \{0, 1, \perp\}$ is output in $\Pi_{\text{RBC}}^{(j,2)}$, set $V_i[v] = V_i[v] + 1$ if the following hold.
 - * $v \in \{0, 1\}$ and $U_i[v] \geq n - t$.
 - * $v = \perp$ and $U_i[0] \geq 1$ and $U_i[1] \geq 1$.
 - Once $V_i[0] + V_i[1] + V_i[\perp] = n - t$, set u_i to be
 - * $b \in \{0, 1\}$, if $V_i[b] \geq 1$
 - * $\text{Coin}(\text{iter})$, otherwise.
 - Once $V_i[b] \geq n - t$ for some $b \in \{0, 1\}$, output b .

^a The variables U_i and V_i used for every iteration are *different* local variables that are not overwritten in subsequent iterations. For readability, we omit writing the iteration number for these variables.

Fig. 4: Code for P_i having input b_i to participate in an EVBA protocol with externally valid view-based predicate Q

Lemma 12. *For two honest parties P_i and P_j and any iteration iter ,*

- (a) *If $U_i[u] \geq n - t$ and if $u' \neq u$, then $U_j[u'] < n - t$.*
- (b) *If $V_i[v] \geq n - t$ and if $v' \neq v$, then $V_j[v'] < n - t$.*

Proof. We prove part (a); part (b) follows by the same reasoning. For contradiction, assume that $U_j[u'] \geq n - t$. Let $\mathcal{P}_i$ be the set of parties P_k for which P_i outputs u in $\Pi_{\text{RBC}}^{(k,1)}$. Similarly, let $\mathcal{P}_j$ be the set of parties P_k for which P_j outputs u' in $\Pi_{\text{RBC}}^{(k,1)}$. Note that $|\mathcal{P}_i \cap \mathcal{P}_j| \geq n - 2t \geq 1$. Thus, $\mathcal{P}_i \cap \mathcal{P}_j$ contains at least one party, say P_ℓ . This violates the consistency of $\Pi_{\text{RBC}}^{(\ell,1)}$. $\square$

The following Lemma follows from inspection of Figure 4 and Lemma 12.

Lemma 13. *Consider honest parties P_i and P_j . For any iteration iter and $b \in \{0, 1\}$, if $V_i[b] \geq 1$, then $V_j[1 - b] = 0$.*

Lemma 14 (Agreement). *If any honest party outputs b , then all honest parties output b .*

Proof. Let P_i be the first party that outputs b , say during iteration iter , implying that $V_i[b] \geq n - t$. We prove that every party P_j outputs either in iteration iter or iteration $\text{iter} + 1$. From the consistency of Π_{RBC} , during iteration iter , since $V_i[b] = n - t$, $V_j[b]$ also eventually becomes $n - t$. Thus, if P_j outputs in iteration iter , then it outputs b .

Now, suppose that P_j instead continues to participate in iteration $\text{iter} + 1$. From Lemma 12, $V_j[\perp]$ is always less than $n - t$, and all P_j set u_j to be b for iteration $\text{iter} + 1$. During $\text{iter} + 1$, $U_j[1 - b]$ is always 0. From inspection of Figure 4, this also implies that both $V_j[1 - b]$ and $V_j[\perp]$ are always 0. Thus, if P_j sets any value for u_j at the end of iteration $\text{iter} + 1$, then it does not proceed to the next iteration and outputs b in iteration $\text{iter} + 1$. $\square$

Lemma 15 (External Validity). *If any honest party P_i outputs b , then $Q(\text{view}_i, b)$ holds true.*

Proof. If both bits are valid, then external validity follows from agreement. Suppose that only one bit b is valid. From inspection of Figure 4, we make the following observations. For any party P_i , only $U_i[b] > 0$, and $U_i[1 - b]$ is always 0. Also, only $V_i[b] > 0$, and $V_i[\perp]$ is always 0. Thus, if P_i outputs, then it outputs b during the first iteration. $\square$

Lemma 16. *Consider any honest party P_i . The following hold for every iteration iter .*

1. *If each honest P_i participates in iter with some value u_i , then each honest P_i reaches the end of the iteration and sets some value for u_i .*
2. *We say that any iteration $\text{iter} > 1$ is lucky if each honest party P_i participates with $u_i = \text{Coin}(\text{iter} - 1) = b$. If iteration iter is lucky, then all parties output b in iteration iter .*

Proof. The proof of part 1 follows from inspection of Figure 4, since each honest party broadcasts its respective u_i and v_i values. Consider iter to be lucky. From inspection of Figure 4, only $U_i[b] > 0$, $U_i[1 - b]$ remains 0. Further, only $V_i[b] > 0$, $V_i[\perp]$ is always 0. Now, consider two cases.

1. *No honest party outputs during $\text{iter} - 1$:* Each honest P_i participates in iter with input $u_i = b$, and $V_i[b]$ eventually grows to $n - t$. Thus, all parties output b .
2. *Some honest P_i outputs during $\text{iter}' < \text{iter}$:* From the consistency of Π_{RBC} , since $V_i[b] \geq n - t$ during iteration iter' , $V_j[b]$ also eventually grows to $n - t$. Thus, P_j also outputs b .

$\square$

Lemma 17 (Termination). *If each honest P_i participates with v_i such that $Q(\text{view}_i, v_i)$ holds, then all parties eventually output. If any honest P_i outputs, then all honest P_j eventually output.*

Proof. From Lemma 16, parties keep participating in the protocol until a lucky iteration occurs. Note that during an iteration iter where no party outputs, each honest P_i either sets u_i to be a (common) bit b or $\text{Coin}(\text{iter})$. Thus, after the first iteration, each iteration is lucky with probability $\frac{1}{2}$. The probability that parties do not terminate decreases exponentially in the number of iterations that occur, guaranteeing termination. Now, suppose that party P_i outputs b , say during iteration iter . Then, $V_i[b] \geq n - t$. From the termination of Π_{RBC} , $V_j[b]$ eventually grows to $n - t$ for each P_j . $\square$

Cost: Each iteration makes $O(n)$ calls to Π_{RBC} (for short messages), each of which incurs a communication cost of $O(n^4\lambda)$ bits. Note that a lucky iteration occurs in 2 expected iterations, implying that, in expectation, it takes just 3 iterations for parties to output. Thus, the expected communication complexity is $O(n^5\lambda)$ bits. This also implies that, in expectation, Π_{EVBA} incurs $O(D)$ synchronous rounds and $O(1)$ asynchronous rounds.

7.2 Agreement on a Common Subset

An Agreement on a Common Subset (ACS) protocol is used to agree on a set of values input by a subset of at least $n - t$ parties. Each value in the output satisfies a view-based predicate Q . We use a standard construction in the literature [BOCG93]: each party broadcasts its input, and n parallel instances of Π_{EVBA} are run where each instance of Π_{EVBA} is used to agree on whether that party's input should be included in the final output. A party inputs 1 to $\Pi_{\text{EVBA}}^{(j)}$ on receiving P_j 's input from its broadcast and checking that it is valid. Once $n - t$ instances of Π_{EVBA} terminate with output 1, a party inputs 0 to the rest of the Π_{EVBA} instances. Once all instances of Π_{EVBA} terminate, the instances that output 1 determine the output: the (greater than $n - t$) inputs broadcasted by these parties will eventually be heard by all parties and can be output.

We remark that while existing constructions for ACS assume that $n > 3t$ holds, we are able to adopt the above standard construction because $n > 3t$ is required only to construct Π_{EVBA} , and we have constructed Π_{EVBA} with $n > 2t$ under granular synchrony. We present detailed steps of Π_{ACS} in Figure 9.

On removing the assumption of a common coin in Π_{EVBA} : A strong common coin can be instantiated using a threshold secret-shared key. To generate a threshold secret-shared key, each party secret shares a value, and by aggregating $t + 1$ shares, each party locally obtains its share of the key. The difficulty lies in having the parties *agree* on this set of $t + 1$ secrets, which itself requires ACS, leading to a circular problem.

We borrow a technique from [DYX⁺22] that circumvents this by having each P_i broadcast this set of $t + 1$ parties for instance $\Pi_{\text{EVBA}}^{(i)}$. Notice that Π_{EVBA} has an additional *Good-Case-Coin-Free* property: if only a single input is externally valid, then termination is guaranteed even *without* a common coin. This helps ensure that $\Pi_{\text{EVBA}}^{(i)}$ terminates even when P_i is corrupt and does not broadcast anything.

Cost: Π_{ACS} incurs a communication cost of $O(n^3 \lambda \cdot |v| + n^6 \lambda)$ bits, where $|v|$ denotes the maximum size of an input to Π_{ACS} . This is due to invoking n instances of Π_{RBC} and Π_{EVBA} . In expectation, Π_{ACS} incurs $O(D \log n)$ synchronous rounds and $O(\log n)$ asynchronous rounds ⁶.

8 Multi Party Computation

Our MPC protocol computes functions expressed as arithmetic circuits over $\mathbb{F}$. While addition gates can be evaluated locally, multiplication gates are evaluated using multiplication triples generated in a pre-processing phase. We use the framework of [CP16] to generate these triples.

The protocols used in our pre-processing phase center around an Agreement on Share Providers (ASP) protocol that agrees on a set of parties that have each secret shared a multiplication triple. The agreed upon set is a synchronously extended set $\mathcal{CS}$ of size at least $n - t$ that contains at least $t + 1$ honest parties. Using such a set $\mathcal{CS}$ of multiplication triples shared by at least $t + 1$ honest parties, a linear combination of the triples shared by parties in $\mathcal{CS}$ can be used to obtain secret shares of multiplication triples that are *random* to the adversary.

We present our ASP protocol in §8.1 and give a detailed overview of the pre-processing phase of our MPC protocol in §8.2. We present our MPC protocols in §E.

The ASP protocol is also used to agree on the set of input providers for MPC. Similar to MPC in asynchrony, even in granular synchrony, it is not possible to ensure that the inputs of all honest

⁶ It takes $O(\log n)$ rounds in expectation for the slowest among n parallel instances of Π_{EVBA} to terminate [BOEY03]

parties are taken into consideration while computing the final output. In §E, we show that the number of honest inputs considered by our MPC protocol is optimal.

8.1 Protocol to Agree on a Set of Share Providers

Each party participates in an Agreement on Share Providers (ASP) protocol with a private input to agree on a common set $\mathcal{CS}$ that contains a *synchronously extended* set of at least $n - t$ parties that have secret shared their inputs; such a set is guaranteed to contain at least $t + 1$ honest parties.

Recall that in Π_{Collect} , a collection that contains inputs from a synchronously extended set of parties of size $n - t$ was constructed by having each party propose a set that includes the inputs of parties synchronously connected to it. Here, we make each party propose a set that instead includes synchronously connected parties that have *secret shared* their inputs.

In Π_{Collect} , this was done by having each party wait for $D\Delta$ time to receive the inputs of parties synchronously connected to it. Similarly, we upgrade our VSS protocol to have an additional property we call *timely verification*: once an honest dealer initiates an instance of Π_{VSS} , all honest parties that are synchronously connected to it locally set an intermediate boolean value to true within time $D\Delta$. On setting this value, an honest party eventually obtains its share, or learns that D is either corrupt, or *partitioned* from it.

Formally, we modify Π_{VSS} to obtain Π_{VSS}^+ that provides the following additional property of *timely verification*. To achieve *timely verification* in Π_{VSS} , we also modify Π_{RBC} to obtain Π_{RBC}^+ that provides *timely verification*.

Timely Verification for VSS: In Π_{VSS}^+ , each P_i additionally outputs an intermediate value int_i and a value flag_i that is 1 if either D is corrupt or if $P_i \not\leftrightarrow D$. If any P_i outputs $\text{int}_i = 1$, then P_i either outputs a value for s_i or outputs $\text{flag}_i = 1$. If $P_i \leftrightarrow D$ and D is honest, then P_i sets $\text{int}_i = 1$ within time $D\Delta$.

Timely Verification for RBC: In Π_{RBC}^+ , each P_i additionally outputs an intermediate value int_i and an indication corrupt_i of whether S is corrupt. If any party P_i outputs $\text{int}_i = 1$, then P_i either outputs a value m_i or outputs $\text{corrupt}_i = 1$. If $P_i \leftrightarrow S$ and S is honest, then it sets $\text{int}_i = 1$ within time $D\Delta$.

It is straightforward to modify Π_{RBC} to achieve timely verification: P_i can set int_i to be 1 on receiving a message and its hash $((\langle m \rangle_S, \langle h \rangle_S))$, where $h = H(m)$ from S . Since this message is relayed by every party, if no equivocating messages from S are seen, then the termination of Π_{RBC}^+ is guaranteed. Otherwise, the equivocation evidence (which is also relayed) indicates that S is corrupt. We present Π_{RBC}^+ that achieves *timely verification* in Appendix B.

The timely verification of Π_{RBC}^+ enables the timely verification of Π_{VSS}^+ . On setting int_i to 1 in Π_{RBC}^+ used by D to broadcast the set of encrypted shares, P_i can be sure that all parties either receive this set, or identify that D is corrupt.

If D is not identified to be corrupt, then honest parties participate in Π_{Collect} with OK or NOK. Once Π_{Collect} terminates, D broadcasts the shares of parties not included in the collection that it outputs. Suppose that P_i outputs a collection at time T . If $P_i \leftrightarrow D$, then it expects to set int_i to 1 in this broadcast within time $T + 2D\Delta$. This follows from the timely termination of Π_{Collect} and the timely verification of Π_{RBC}^+ . If this flag is not set, then D is either corrupt or not connected to P_i . Otherwise, all parties receive these shares from D , verify that they are correct using the broadcasted commitment, and obtain an output. Figure 3 presents Π_{VSS}^+ , with lines in blue highlighting the extra steps required for timely verification.

In §C, we present a batched VSS protocol allowing a dealer to share L secrets, and we prove that it similarly provides timely verification.

Protocol $\Pi_{\text{ASP}}(s_i)$

- Initialize set $\mathcal{N}_i$ to $\emptyset$. Participate in $\Pi_{\text{VSS}}^{+(i)}(P_i, s_i)$.
- On setting int_i to 1 within time $n\Delta$ in $\Pi_{\text{VSS}}^{+(j)}$, add P_j to $\mathcal{N}_i$.
- On setting flag_i to 1 in $\Pi_{\text{VSS}}^{+(j)}$, remove P_j from $\mathcal{N}_i$.
- Define $Q(\text{view}, \mathcal{N})$ to be true if view contains an output from $\Pi_{\text{VSS}}^{+(j)}$ for each $P_j \in \mathcal{N}$.
- On receiving s_i as output in $\Pi_{\text{VSS}}^{+(j)}$ for all $P_j \in \mathcal{N}_i$, participate in $\Pi_{\text{ACS}}(Q, \mathcal{N}_i)$.
- Let $\mathbf{v}$ be the output of Π_{ACS} . Output $\mathcal{CS} = \cup_{\mathcal{N}_j \in \mathbf{v}} \mathcal{N}_j$

Fig. 5: Code for party $P_i \in \mathcal{P}$ having input s_i to agree on a common subset $\mathcal{CS}$ of $n - t$ input providers that includes parties synchronously connected to them.

Protocol Π_{ASP} : Now, we can design Π_{ASP} as follows. Each party P_j shares its input through an instance $\Pi_{\text{VSS}}^{+(j)}$. P_i includes all parties P_j for which int_i was set to 1 during $\Pi_{\text{VSS}}^{+(j)}$ in a local $\mathcal{N}_i$ set. This set is guaranteed to include all *honest parties synchronously connected* to P_i . On identifying that a particular party is either corrupt or not synchronously connected to it, P_i simply removes that party from $\mathcal{N}_i$. Once the respective $\Pi_{\text{VSS}}^{+(j)}$ instances have completed for *all* parties in $\mathcal{N}_i$, each P_i participates in an instance of Π_{ACS} with input $\mathcal{N}_i$ to output a common subset of $n - t$ inputs $\mathbf{v}$. $\mathcal{CS}$ is then computed by taking the union of sets present in $\mathbf{v}$. We present Π_{ASP} in Figure 5.

Lemma 18. *In Π_{ASP} , all honest parties participate with input s_i output a common set $\mathcal{CS}$ that contains a synchronously extended set of size at least $n - t$. For each party $P_i \in \mathcal{CS}$, each party P_j holds a point $p(j)$ on some $(n - t - 1)$ -degree polynomial $p(\cdot)$. If P_i is honest, then $p(0) = s_i$.*

Proof. From the consistency of Π_{VSS}^{+} , Q defined in Figure 5 is a view-based predicate. The validity of Π_{ACS} ensures that $\mathcal{CS}$ is a common subset of $\mathcal{P}$, and that $|\mathcal{CS}| \geq n - t$. From the *timely verification* of Π_{VSS}^{+} , each honest $P_i \in \mathcal{CS}$ broadcasts $\mathcal{N}_i$ containing all parties in $\text{Extended}(\{P_i\})$. Thus, $\text{Extended}(\mathcal{CS}) \subseteq \mathcal{CS}$. The proof follows from the *consistency* and *correctness* of Π_{VSS}^{+} . $\square$

Lemma 19. *The view of $\mathcal{A}$ is independent of s_i for each $P_i \in \mathcal{CS}$.*

Proof. Follows from the privacy of Π_{VSS}^{+} . $\square$

Cost: The cost of invoking Π_{ACS} is $O(n^6 \lambda)$ bits. To agree on a set of parties that have shared L secrets in $\mathbb{F}$, the cost of invoking n instances of Π_{VSS}^{+} will instead be $O(n^4 \log |\mathbb{F}| \cdot L + n^5 \lambda)$, rendering the cost of Π_{ASP} to be $O(n^4 \log |\mathbb{F}| \cdot L + n^6 \lambda)$ bits. Π_{ASP} incurs $O(D \log n)$ synchronous and $O(\log n)$ asynchronous rounds in expectation, which follows from the round complexity of Π_{ACS} .

8.2 Overview of Pre-processing Phase

To perform MPC, we use the standard approach of evaluating a circuit consisting of linear and non-linear (multiplication) gates. An instance of Π_{ASP} is used to agree on a common subset of parties that have secret-shared their inputs. Given shares of inputs to a linear gate, shares of the output can be computed locally owing to the linearity of Π_{VSS} . To compute shares of the output of multiplication gates, we follow Beaver’s circuit-randomization method [Bea92] that uses a function-independent pre-processing phase Π_{Prep} to generate c_M Beaver triples⁷ that are uniformly random. These can later be used in a function-dependent online phase Π_{CircEval} that evaluates the function by evaluating a circuit.

⁷ A Beaver triple is a triple (a, b, c) , where $c = a \cdot b$. This triple is considered uniformly random if a and b are uniformly random in $\mathcal{A}$ ’s view.

Verifiably sharing multiplication triples: Π_{Prep} is based on a protocol Π_{TripSh} that identifies a common set $\mathcal{CS}$ that contains a synchronously extended set of $n - t$ parties that have secret shared c_M multiplication triples. First, an instance of Π_{ASP} is used to identify a set of parties that have shared *some* triples.

To verify that the triples shared are *multiplication* triples, we use a well-known technique employed by prior works [DPSZ12, CP23]. For each multiplication triple (a, b, c) that is shared by a party, the party additionally shares a second multiplication triple (x, y, z) . By checking that $r(ab - c) = z - xy$ for some uniformly random r that was unknown while the triples were shared, it can be verified that (a, b, c) is indeed a multiplication triple.

The pre-processing phase: Since $\mathcal{A}$ knows the triples shared by corrupt parties in $\mathcal{CS}$, the triples obtained from Π_{TripSh} cannot directly be used as Beaver triples. To obtain multiplication triples which are *uniformly random* in $\mathcal{A}$'s view, we follow the framework of [CP16] that transforms the triples $\{(\llbracket a_\ell^j \rrbracket, \llbracket b_\ell^j \rrbracket, \llbracket c_\ell^j \rrbracket)\}_{j \in \mathcal{CS}}$ obtained from Π_{TripSh} into triples $\{(\llbracket \mathbf{a}_\ell^j \rrbracket, \llbracket \mathbf{b}_\ell^j \rrbracket, \llbracket \mathbf{c}_\ell^j \rrbracket)\}_{j \in \mathcal{CS}}$ that lie on polynomials $A_\ell(\cdot)$, $B_\ell(\cdot)$ and $C_\ell(\cdot)$ of degrees t , t and $2t$ respectively, for each $\ell \in [c_M]$.

Note that each $C(\cdot)_\ell$, being of degree $2t$, has to be defined using $2t + 1$ points. Since $|\mathcal{CS}|$ may be less than $2t + 1$, we use a trick presented in [BCV24] that adds dummy triples to $\mathcal{CS}$ to increase its size to $2t + 1$. Since $\mathcal{CS}$ has at least $t + 1$ honest parties, $\mathcal{A}$ learns at most t transformed points and does not learn $A_\ell(\cdot)$, $B_\ell(\cdot)$ and $C_\ell(\cdot)$. By evaluating these polynomials at a point that was not used previously, parties obtain c_M Beaver triples.

We present protocols Π_{Beaver} , Π_{TripSh} , Π_{Prep} and Π_{CircEval} in §E, and state the properties that our final Π_{CircEval} protocol achieves below.

Theorem 1. *Each P_i participates in Π_{CircEval} to achieve the following, except with negligible probability. (1) Each P_i outputs a set $\mathcal{CS}$ that contains a synchronously extended set of size at least $n - t$, and a value $y = F(x_1, \dots, x_n)$, where $x_j = \perp$ if $P_j \notin \mathcal{CS}$. (2) The view of $\mathcal{A}$ is independent of x_i for each honest P_i . Π_{CircEval} incurs a communication of $O(n^4 \log |\mathbb{F}| \cdot c_M + n^6 \lambda)$ and $O(n^2 \lambda \cdot c_M)$ bits in the offline and online phases, respectively, and a round complexity of $O(D \log n)$ synchronous and $O(\log n)$ asynchronous rounds in expectation.*

References

- [AC23] Ananya Appan and Ashish Choudhury. Network agnostic mpc with statistical security. In *Theory of Cryptography Conference*, pages 63–93. Springer, 2023.
- [BCLZL23] Renas Bacho, Daniel Collins, Chen-Da Liu-Zhang, and Julian Loss. Network-agnostic security comes (almost) for free in dkg and mpc. In *Annual International Cryptology Conference*, pages 71–106. Springer, 2023.
- [BCV24] Nidhish Bhimrajka, Ashish Choudhury, and Supreeth Varadarajan. Network-agnostic multi-party computation revisited. In *IACR International Conference on Public-Key Cryptography*, pages 171–204. Springer, 2024.
- [Bea92] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology—CRYPTO’91: Proceedings 11*, pages 420–432. Springer, 1992.
- [BKP11] Michael Backes, Aniket Kate, and Arpita Patra. Computational verifiable secret sharing revisited. In *Advances in Cryptology—ASIACRYPT 2011: 17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, South Korea, December 4–8, 2011. Proceedings 17*, pages 590–609. Springer, 2011.
- [BLZL20] Erica Blum, Chen-Da Liu-Zhang, and Julian Loss. Always have a backup plan: fully secure synchronous mpc with asynchronous fallback. In *Annual International Cryptology Conference*, pages 707–731. Springer, 2020.

- [BOCG93] Michael Ben-Or, Ran Canetti, and Oded Goldreich. Asynchronous secure computation. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, pages 52–61, 1993.
- [BOEY03] Michael Ben-Or and Ran El-Yaniv. Resilient-optimal interactive consistency in constant time. *Distributed Computing*, 16(4):249–262, 2003.
- [BOGW19] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*, pages 351–371. 2019.
- [BOKR94] Michael Ben-Or, Boaz Kelmer, and Tal Rabin. Asynchronous secure computations with optimal resilience. In *Proceedings of the thirteenth annual ACM symposium on Principles of distributed computing*, pages 183–192, 1994.
- [CGMA85] Benny Choc, Shafi Goldwasser, Silvio Micali, and Baruch Awerbuch. Verifiable secret sharing and achieving simultaneity in the presence of faults. In *Annual Symposium on Foundations of Computer Science (Proceedings)*, pages 383–395, 1985.
- [CKLS02] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Strobil. Asynchronous verifiable secret sharing and proactive cryptosystems. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*, pages 88–97, 2002.
- [CKS00] Christian Cachin, Klaus Kursawe, and Victor Shoup. Random oracles in constantipole: practical asynchronous byzantine agreement using cryptography. In *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing*, pages 123–132, 2000.
- [CMSK07] Byung-Gon Chun, Petros Maniatis, Scott Shenker, and John Kubiawicz. Attested append-only memory: Making adversaries stick to their word. *ACM SIGOPS Operating Systems Review*, 41(6):189–204, 2007.
- [Coh16] Ran Cohen. Asynchronous secure multiparty computation in constant time. In *Public-Key Cryptography–PKC 2016*, pages 183–207. Springer, 2016.
- [CP16] Ashish Choudhury and Arpita Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Transactions on Information Theory*, 63(1):428–468, 2016.
- [CP23] Ashish Choudhury and Arpita Patra. On the communication efficiency of statistically secure asynchronous mpc with optimal resilience. *Journal of Cryptology*, 36(2):13, 2023.
- [DI05] Ivan Damgård and Yuval Ishai. Constant-round multiparty computation using a black-box pseudorandom generator. In *Annual International Cryptology Conference*, pages 378–394. Springer, 2005.
- [DLS88] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the presence of partial synchrony. *Journal of the ACM (JACM)*, 35(2):288–323, 1988.
- [DN03] Ivan Damgård and Jesper Buus Nielsen. Universally composable efficient multiparty computation from threshold homomorphic encryption. In *Annual international cryptology conference*, pages 247–264. Springer, 2003.
- [DPSZ12] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Annual Cryptology Conference*, pages 643–662. Springer, 2012.
- [DWZ23] Sisi Duan, Xin Wang, and Haibin Zhang. Fin: practical signature-free asynchronous common subset in constant time. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 815–829, 2023.
- [DXT⁺23] Sourav Das, Zhuolun Xiang, Alin Tomescu, Alexander Spiegelman, Benny Pinkas, and Ling Ren. Verifiable secret sharing simplified. *Cryptology ePrint Archive*, 2023.
- [DYX⁺22] Sourav Das, Thomas Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous distributed key generation. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2518–2534. IEEE, 2022.
- [Fel87] Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*, pages 427–438. IEEE, 1987.
- [FLP85] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM (JACM)*, 32(2):374–382, 1985.

- [GAC⁺24] Neil Giridharan, Ittai Abraham, Natacha Crooks, Kartik Nayak, and Ling Ren. Granular synchrony. *arXiv preprint arXiv:2408.12853*, 2024.
- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game, or a completeness theorem for protocols with honest majority. In *STOC*. 1987.
- [GPS19] Yue Guo, Rafael Pass, and Elaine Shi. Synchronous, with a chance of partition tolerance. In *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part I 39*, pages 499–529. Springer, 2019.
- [GRR98] Rosario Gennaro, Michael O Rabin, and Tal Rabin. Simplified vss and fast-track multiparty computations with applications to threshold cryptography. In *Proceedings of the seventeenth annual ACM symposium on Principles of distributed computing*, pages 101–111, 1998.
- [HNP05] Martin Hirt, Jesper Buus Nielsen, and Bartosz Przydatek. Cryptographic asynchronous multiparty computation with optimal resilience. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 322–340. Springer, 2005.
- [KN23] Simon Holmgård Kamp and Jesper Buus Nielsen. Byzantine agreement decomposed: Honest majority asynchronous atomic broadcast from reliable broadcast. *Cryptology ePrint Archive*, 2023.
- [KZG10] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16*, pages 177–194. Springer, 2010.
- [LM07] Jinyuan Li and David Mazieres. Beyond one-third faulty replicas in byzantine fault tolerant systems. In *NSDI*, pages 10–10, 2007.
- [LSP82] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. In *ACM Transactions on Programming Languages and Systems 4, 3 (1982)*, page 382–401. 1982.
- [MXC⁺16] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. The honey badger of bft protocols. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 31–42, 2016.
- [Ors21] Emmanuela Orsini. Efficient, actively secure mpc with a dishonest majority: a survey. In *Arithmetic of Finite Fields: 8th International Workshop, WAIFI 2020, Rennes, France, July 6–8, 2020, Revised Selected and Invited Papers 8*, pages 42–71. Springer, 2021.
- [Ped91] Torben Pnyds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.
- [RBO89] Tal Rabin and Michael Ben-Or. Verifiable secret sharing and multiparty protocols with honest majority. In *Proceedings of the twenty-first annual ACM symposium on Theory of computing*, pages 73–85, 1989.
- [Sha79] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [SS24] Victor Shoup and Nigel P Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. *Journal of Cryptology*, 37(3):27, 2024.
- [Tou84] Sam Toueg. Randomized byzantine agreements. In *Proceedings of the third annual ACM symposium on Principles of distributed computing*, pages 163–178, 1984.
- [VCB⁺11] Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, Lau Cheuk Lung, and Paulo Verissimo. Efficient byzantine fault-tolerance. *IEEE Transactions on Computers*, 62(1):16–30, 2011.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.
- [YLF⁺21] Thomas Yurek, Licheng Luo, Jaiden Fairoze, Aniket Kate, and Andrew Miller. hbacss: How to robustly share many secrets. *Cryptology ePrint Archive*, 2021.

A Network Condition for Granular Synchrony

A.1 Equivalence with [GAC⁺24]

In this section, we show that the condition on the communication graph that we assume is equivalent to the condition on the communication graph assumed by [GAC⁺24]. We first recall the condition that we assume.

Condition 1 For any set Corr of t parties, $\forall \text{Hon}_1, \text{Hon}_2 \subseteq \mathcal{P} \setminus \text{Corr}$, it is not possible that Hon_1 and Hon_2 are partitioned from each other while $|\text{Hon}_1| \geq n - 2t$ and $|\text{Hon}_2| \geq n - 2t$.

We next present the condition assumed by [GAC⁺24].

Condition 2. For any set Corr of size t , $\forall \text{Hon}_1 \subseteq \mathcal{P} \setminus \text{Corr}$ with $|\text{Hon}_1| \geq n - 2t$, $|\text{Extended}(\text{Hon}_1)| \geq t + 1$.

We now prove that both conditions are equivalent.

Theorem 2. Condition 1 holds if and only if Condition 2 holds.

Proof. Let Corr be any set of corrupt parties of size *exactly* t . We show that, in this case, Conditions 1 and 2 are equivalent. It is enough to consider only this case since it never hurts the adversary to corrupt more parties (and make them behave honestly if needed).

1. Condition 1 $\rightarrow$ Condition 2: Consider a set $\text{Hon}_1 \in \text{Hon}$ of size $n - 2t$. $\text{Hon}_2 = \text{Hon} \setminus \text{Extended}(\text{Hon}_1)$ is a set of honest parties partitioned from Hon_1 , and has size $n - t - |\text{Extended}(\text{Hon}_1)|$. Since $|\text{Hon}_2| < n - 2t$, we must have $|\text{Extended}(\text{Hon}_1)| > t$.
2. Condition 2 $\rightarrow$ Condition 1: Consider a set $\text{Hon}_1 \in \text{Hon}$ of size $n - 2t$. For a set Hon_2 of honest parties to be partitioned from Hon_1 , Hon_2 must be disjoint from $\text{Extended}(\text{Hon}_1)$. Since $|\text{Extended}(\text{Hon}_1)| \geq t + 1$, $|\text{Hon}_2| \leq n - t - (t + 1) < n - 2t$. $\square$

A.2 Necessity for RBC

Here, we prove that if Condition 1 does not hold, then for certain designated senders, no protocol Π achieves RBC in granular synchrony. Since a protocol for VSS can be used to construct a protocol for RBC, this implies that Condition 1 is necessary for VSS as well.

Theorem 3. Under granular synchrony, if Condition 1 does not hold, then there exists a sender S for which no protocol Π for RBC guarantees consistency.

Proof. Let C be any set of t parties. For contradiction, consider sets $X \subset \mathcal{P} \setminus C$ and $Y \subset \mathcal{P} \setminus C$ that are partitioned from each other and are both of sizes greater than $n - 2t$. We can then find sets A and B of sizes greater than $n - 2t$ for which $A \cup B \cup C = \mathcal{P}$ as follows. For each party P_i in Hon :

- If $P_i \in \text{Extended}(X)$, add P_i to A .
- If $P_i \in \text{Extended}(Y)$, add P_i to B .
- Otherwise, add P_i to A .

Note that since $|B| \geq n - 2t$, $|A| = |\mathcal{P} \setminus (C \cup B)| \leq n - (t + n - 2t) = t$. Similarly, $|B| \leq t$. We prove that for any sender S that belongs to C , there is an attack that violates consistency. Consider the following three executions.

- *Execution 1*: All parties in B are corrupt and crash. S has input 1. From the validity of Π , all parties in A eventually output 1, say at time T_1 .
- *Execution 2*: All parties in A are corrupt and crash. S has input 0. From the validity of Π , all parties in B eventually output 0, say at time T_2 .
- *Execution 3*: All parties in C , including S , are corrupt. All messages between parties in A and parties in B are delayed for $\max(T_1, T_2)$ time. Parties in C interact with parties in A similar to *Execution 1*, and with parties in B similar to *Execution 2*.

Consider *Execution 3*. Since the view of parties in A is exactly like *Execution 1*, all parties in A output 1. Similarly, since the view of parties in B is exactly like *Execution 2*, all parties in B output 0. This, being a violation of consistency, results in a contradiction. $\square$

B Reliable Broadcast for Long Messages

In this section we present Π_{RBC}^+ : an RBC protocol for long messages that additionally guarantees *timely verification* (which is required for our ACS protocol). §5 and §8.1 present the intuition behind our ideas used to reduce communication cost for long messages and provide timely verification respectively. We reduce communication by having parties participate with the hash of the message as input to Π_{Collect} . We ensure timely verification by having parties identify S to be corrupt if they see two different hash values signed by S .

In Figure 6, Π_{Collect} is guaranteed timely participation since each party relays the first message (and hash) that it sees signed by S . We formally prove the properties of Π_{RBC}^+ below.

Lemma 20. Q_{in} defined in Figure 6 has timely validation.

Proof. Follows since the definition of Q_{in} does not depend on the party's view. $\square$

Lemma 21 (Validity). When S is honest, all parties eventually output m .

Proof. Since S is honest, S sends $(\langle m \rangle_S, \langle h \rangle_S)$ to all parties. Each honest party P_i participates in Π_{Collect} with input $h = H(m)$. Since S sends a single $(\langle m \rangle_S, \langle h \rangle_S)$ to all parties, no P_i sets $\text{corrupt}_i = 1$ and only $Q_{\text{in}}(\text{view}_i, \langle h \rangle_S)$ is true. From the termination of Π_{Collect} , each P_i eventually outputs $\mathcal{C}_i$ containing only $\langle \langle h \rangle_S \rangle_*$. Since P_i eventually receives $\langle m \rangle_S$ from S where $H(m) = h$, P_i outputs m . $\square$

Lemma 22 (Consistency). If any party outputs m' , then all parties that output anything output m' .

Proof. Consider party P_i that outputs $m_i = m'$. This means that P_i has seen $\mathcal{C}$ containing only $\langle \langle h' \rangle_S \rangle_*$, where $h' = H(m')$. Consider any party P_j that outputs $m_j = m''$. P_j has seen $\mathcal{C}'$ containing only $\langle \langle h'' \rangle_S \rangle_*$, where $h'' = H(m'')$. From Corollary 5, $\mathcal{C} \cap \mathcal{C}'$ contains at least one honest party. Since each honest party participates in Π_{Collect} with a single input, this means that $H(m') = H(m'')$. From the collision resistance of H , this means that P_j outputs $m_j = m'$. $\square$

Lemma 23 (Timely Termination). If any party P_i outputs at time T and $P_j \leftrightarrow P_i$, then P_j outputs by time $T + D\Delta$.

Proof. Suppose P_i outputs m' at time T . This means that P_i has (1) output $\mathcal{C}_i$ containing only $\langle \langle h \rangle_S \rangle_*$ and (2) received $\langle m' \rangle_S$ such that $h = H(m')$. The timely termination of Π_{Collect} guarantees that each P_j outputs $\mathcal{C}_j$ by time $T + D\Delta$ if $P_i \leftrightarrow P_j$. Since each party on the path from P_i to P_j further relays m' (which is unique, as argued for consistency), P_j receives and outputs m' by $T + D\Delta$. $\square$

Protocol $\Pi_{\text{RBC}}^+(\mathcal{S}, m)$

- Each $P_i \in \mathcal{P}$ defines the following predicates.
 - $Q_{\text{in}}(\text{view}, v)$ is defined to be true if v is of the form $\langle h \rangle_{\mathcal{S}}$ (i.e., there is a valid signature). (i.e., there is a valid signature).
 - $Q_{\text{out}}(\mathcal{C})$ is true if $\mathcal{C}$ contains signatures on the *same* value.
- $\mathcal{S}$ does the following: On having input m , compute $h = H(m)$, and send $(\langle m \rangle_{\mathcal{S}}, \langle h \rangle_{\mathcal{S}})$ to all parties in $\mathcal{P}$.
- Each $P_i \in \mathcal{P}$ does the following :
 - On receiving $(\langle m \rangle_{\mathcal{S}}, \langle h \rangle_{\mathcal{S}})$ such that $H(m) = h$ for the first time, **set int_i to 1**, and **send $(\langle m \rangle_{\mathcal{S}}, \langle h \rangle_{\mathcal{S}})$ to all parties in $\mathcal{P}$** .
 - Participate in $\Pi_{\text{Collect}}(Q_{\text{in}}, Q_{\text{out}})$ with input $\langle h \rangle_{\mathcal{S}}$.
 - On receiving equivocating messages $\langle h \rangle_{\mathcal{S}}$ and $\langle h' \rangle_{\mathcal{S}}$ where $h \neq h'$, **set corrupt_i to 1**. **If equivocating messages have been received for the first time, send $\langle h \rangle_{\mathcal{S}}$ and $\langle h' \rangle_{\mathcal{S}}$ to all parties in $\mathcal{P}$** .
 - On obtaining output $\mathcal{C}_i$ containing inputs of the form $\langle h \rangle_{\star}$, on receiving $\langle m \rangle_{\mathcal{S}}$ such that $H(m) = h$, **send $\langle m \rangle_{\mathcal{S}}$ to all parties and output m** .

Fig. 6: Reliable broadcast protocol for sender $\mathcal{S}$ for long messages. Steps colored gray are used to ensure that *timely participation* and *timely termination* hold. Steps colored blue are used for *timely verification*.

Lemma 24 (Timely Verification). *If party P_i outputs $\text{int}_i = 1$, then P_i either outputs a value m_i or outputs $\text{corrupt}_i = 1$. If $P_i \leftrightarrow \mathcal{S}$ and $\mathcal{S}$ is honest, then it sets $\text{int}_i = 1$ within time $D\Delta$.*

Proof. Suppose party P_i sets int_i to 1. Following what was argued for timely participation, each P_i participates in Π_{Collect} with some input, and outputs $\mathcal{C}_i$. Termination of Π_{Collect} is guaranteed if no honest party P_j sets $\text{corrupt}_j = 1$. If Π_{Collect} terminates, P_i outputs a value for m_i .

Otherwise, some P_j has set $\text{corrupt}_j = 1$. This means that P_j has seen equivocating messages from $\mathcal{S}$. From inspection of Figure 6, each party relays the equivocating evidence. Thus, P_i eventually receives this, and sets $\text{corrupt}_i = 1$.

When $\mathcal{S}$ is honest and $P_i \leftrightarrow \mathcal{S}$, since each party relays $(\langle m \rangle_{\mathcal{S}}, \langle h \rangle_{\mathcal{S}})$, P_i sets int_i to 1 within time $D\Delta$. $\square$

Cost: Each P_i relays the first message signed by $\mathcal{S}$ and the first equivocation evidence that it sees. P_i also relays $\langle m \rangle_{\mathcal{S}}$ where $\mathcal{C}_i$ contains only $\langle \langle h \rangle_{\mathcal{S}} \rangle_{\star}$ and P_i has seen $\langle m \rangle_{\mathcal{S}}$ such that $H(m) = h$. As argued for consistency, there is only one value of m . The total cost of these steps is $O(n^2(|m| + \lambda))$. From the cost of invoking Π_{Collect} , the total communication cost is $O(n^2(|m| + n^2\lambda))$ since each party inputs a hash value of size λ to Π_{Collect} . From the round complexity of Π_{Collect} , Π_{RBC}^+ incurs $O(D)$ synchronous rounds and $O(1)$ asynchronous rounds.

C Verifiable Secret Sharing

C.1 Batching Secrets to Reduce Communication Cost

In this section, we extend Π_{VSS}^+ to allow $\mathcal{D}$ to share L secrets. This brings the communication cost down to $O(n^3\lambda \cdot (L|v| + n))$, where $|v|$ is the size of each secret. We also show that this extended Π_{VSS} allows parties to perform timely verification, i.e., to be able to check if $\mathcal{D}$ is an honest party that is connected to it, within a fixed, known time.

To share L secrets, $\mathcal{D}$ chooses L random $(n - t - 1)$ -degree polynomials and broadcasts encrypted shares of each party, along with a commitment for each polynomial. On checking that *all* received

shares are verified with their respective commitments, a party inputs an OK message to an instance of Π_{Collect} ; otherwise, the party inputs NOK. A collection of messages is guaranteed to include messages from at least $t + 1$ honest parties, and an honest dealer D can safely broadcast the shares of all parties not included in this collection without violating privacy. Since a single instance of Π_{Collect} (which was previously the bottleneck) is used for multiple secrets, the communication cost per secret is brought down to $O(n^3)$. We present our modified Π_{VSS}^+ in Figure 7.

Protocol $\Pi_{\text{VSS}}^+(D, L, \{s^1, \dots, s^L\})$

- Each P_i defines the following predicates.
 - $Q_{\text{in}}(\text{view}, v)$ is true if $v \in \{\text{OK}, \text{NOK}\}$.
 - $Q_{\text{out}}(\mathcal{C})$ is always true.
- D initializes **batchedShares** to $\emptyset$ and does the following for $\ell = 1, \dots, L$.
 - Choose a $n - t - 1$ -degree polynomial $p^\ell(\cdot)$ uniformly at random where $p^\ell(0) = s^\ell$.
 - Run $\text{PC.Commit}(p^\ell(\cdot), n) \rightarrow (\text{com}^\ell, w^\ell)$.
 - Initialize set **shares** $^\ell$ to $\emptyset$. For each $i \in [n]$, do the following and add **shares** $^\ell$ to **batchedShares**.
 - Run $\text{PC.Open}(w, p(\cdot), i) \rightarrow (p^\ell(i), \pi_i^\ell)$.
 - Add $\text{Enc}_{pk_i}(i, p^\ell(i), \pi_i^\ell)$ to **shares** $^\ell$.
- Participate in protocol $\Pi_{\text{RBC}}^{+(1)}(D, (\text{batchedShares}, \text{com}))$
- Each P_i does the following :
 - Set int_i to 1 on setting int_i to 1 in protocol $\Pi_{\text{RBC}}^{+(1)}$.
 - Set flag_i to 1 on setting corrupt_i to 1 in protocol $\Pi_{\text{RBC}}^{+(1)}$.
 - On receiving **batchedShares** as output from $\Pi_{\text{RBC}}^{+(1)}$, if **batchedShares** does not contain L sets of the form $\{(c_1 \dots, c_n), \text{com}\}$, set **flag** $_i = 1$.
 - Set $v_i = \text{OK}$. For $\ell = 1, \dots, L$, let $i, p_i^\ell, \pi_i^\ell = \text{Dec}_{sk_i}(c_i^\ell)$. If $\text{PC.Verify}(\text{com}^\ell, n - t - 1, i, p_i^\ell, \pi_i^\ell) \rightarrow 0$, set $v_i = \text{NOK}$. Participate in $\Pi_{\text{Collect}}(Q)$ with input v_i to obtain output C_i .
- D does the following : Participate in $\Pi_{\text{RBC}}^{+(2)}(\mathcal{C}_D, \{(i, \{(p^\ell(i), \pi_i^\ell)\}_{\ell \in [L]})\}_{\langle \text{OK} \rangle_i \notin \mathcal{C}_D})$.
- Each P_i does the following :
 - Let T be the time at which C_i is obtained as the output of Π_{Collect} . If int_i is not set to 1 in $\Pi_{\text{RBC}}^{+(2)}$ within time $T + 2D\Delta$, set **flag** $_i$ to 1.
 - Set **flag** $_i$ to 1 on setting corrupt_i to 1 in protocol $\Pi_{\text{RBC}}^{+(2)}$.
 - On receiving output from $\Pi_{\text{RBC}}^{+(2)}$, set **flag** $_i$ to 1 if any of the following do not hold.
 - Output is of the form $(\mathcal{C}_D, \{(j, \{(p_j^\ell, \pi_j^\ell)\}_{\ell \in [L]})\}_{\langle \text{OK} \rangle_j \notin \mathcal{C}_D})$ where each value in $\mathcal{C}_D$ is of the form $\langle \text{OK} \rangle_\star$ or $\langle \text{NOK} \rangle_\star$.
 - For each P_j for which $\langle \text{OK} \rangle_j \notin \mathcal{C}_D$, $\text{PC.Verify}(\text{com}^\ell, n - t - 1, j, p_j^\ell, \pi_j^\ell) \rightarrow 1$, for each $\ell = 1, \dots, L$.
 - If the above hold, output $\{s_i^\ell\}_{\ell \in [L]}$ as follows.
 - If $\langle \text{OK} \rangle_i \in \mathcal{C}_D$, output $s_i^\ell = p_i^\ell$ obtained as part of $\Pi_{\text{RBC}}^{+(1)}$.
 - Else, output $s_i^\ell = p_i^\ell$ obtained as part of $\Pi_{\text{RBC}}^{+(2)}$.

Fig. 7: VSS protocol for dealer D with input $\{s^1, \dots, s^L\}$. Lines highlighted in blue are used for timely verification.

We prove that Π_{VSS}^+ provides timely verification. We omit proofs of other properties as they are similar to the case when a single secret is shared.

Lemma 25. *In Π_{VSS}^+ , each P_i outputs an intermediate value int_i and a value flag_i that is true if either D is corrupt or if $P_i \not\leftrightarrow D$. If any P_i outputs $\text{int}_i = 1$, then P_i either outputs L values $\{s_i^1, \dots, s_i^L\}$ or outputs $\text{flag}_i = 1$. If $P_i \leftrightarrow D$ and D is honest, then P_i sets $\text{int}_i = 1$ within time $D\Delta$.*

Proof. Consider an honest P_i that sets int_i to 1. This implies that int_i is set to 1 in $\Pi_{\text{RBC}}^{+(1)}$. From the timely verification of Π_{RBC}^+ , P_i either sets corrupt_i to 1 in Π_{RBC}^+ , or obtains **batchedShares** as output. If **batchedShares** is not of the correct form, then flag_i is set to 1. Otherwise, every honest party eventually obtains **batchedShares** that is of the correct form, and participates in Π_{Collect} with some input.

Π_{Collect} eventually terminates for each party. Let T be the time at which P_i obtained output $\mathcal{C}_i$. If D is honest and $P_i \leftrightarrow D$, from the termination of Π_{Collect} , D outputs $\mathcal{C}_D$ within time $T + D\Delta$. Thus, D initiates $\Pi_{\text{RBC}}^{+(2)}$ within time $T + D\Delta$. From the timely verification of Π_{RBC}^+ , P_i sets int_i to 1 in $\Pi_{\text{RBC}}^{+(2)}$ within time $T + 2D\Delta$. If P_i does not do this, then D is either corrupt, or $P_i \not\leftrightarrow D$, and P_i sets flag_i to 1.

Otherwise, from the timely verification of Π_{RBC}^+ , P_i either sets corrupt_i to 1 in $\Pi_{\text{RBC}}^{+(2)}$ or obtains an output. In the former case, P_i sets flag_i to 1. In the latter case, on receiving an output, P_i either sets flag_i to 1 or to 0. If flag_i is set to 1, then D has not followed the steps of the protocol and is corrupt. If flag_i is set to 0, P_i outputs a value for s_i^ℓ .

Now, suppose that D is honest. From the timely verification of Π_{RBC}^+ , each P_i connected to D sets int_i to 1 in $\Pi_{\text{RBC}}^{+(1)}$ within time $n\Delta$, and therefore sets int_i to 1 in Π_{VSS} within time $D\Delta$. $\square$

Communication Cost : The cost of invoking each instance of Π_{RBC}^+ is $O(n^2(n|v| \cdot L + n^2\lambda))$, where $|v|$ is the size of each secret that is shared. This is because D broadcasts $\mathcal{C}_D$ of size $O(n^2)$ and Ln shares of size $|v|$ (the size of a share equals that of the secret). The cost of invoking Π_{Collect} is $O(n^4\lambda)$ since the input to Π_{Collect} is of constant size. This results in a total communication cost of $O(n^3|v| \cdot L + n^4\lambda)$.

C.2 The Reconstruction Protocol

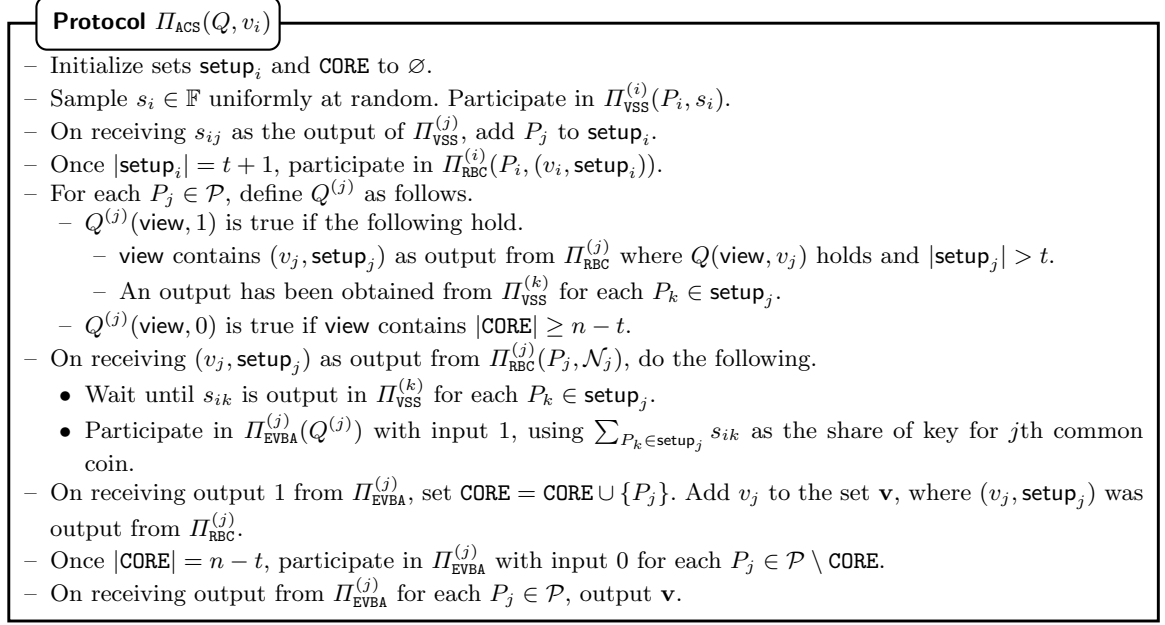
In this section, we present our reconstruction protocol Π_{Rec} that reconstructs a secret s shared through Π_{VSS} . Recall that, after running Π_{VSS} , all honest parties agree on a commitment **com** to this polynomial, and have verified that their shares are correct evaluations of the polynomial. Since there are at least $n - t$ honest parties, the points held by honest parties are sufficient to reconstruct the polynomial. The verification procedure of the polynomial commitment scheme ensures that points shared by corrupt parties that do not lie on the committed polynomial will not be considered while performing the interpolation. We present Π_{Rec} in Figure 8.

Protocol $\Pi_{\text{Rec}}(\llbracket s \rrbracket = (s_i, \pi_i, \text{com}))$

- Initialize set **points** to $\emptyset$. Send s_i, π_i to all parties in $\mathcal{P}$.
- On receiving s_j, π_j from P_j such that $\text{PC.Verify}(\text{com}, j, s_j, \pi_j) \rightarrow 1$, set **points** = **points** $\cup \{s_j\}$.
- Once $|\text{points}| = n - t$, interpolate using Lagrange interpolation to obtain an $(n - t - 1)$ -degree polynomial $p(\cdot)$. Output $s = p(\cdot)$.

Fig. 8: Code for party P_i , who holds a share s_i , to reconstruct the secret s . P_i also inputs a proof π_i that $p(\cdot)$ evaluates to s_i at index i , and a polynomial commitment **com** to $p(\cdot)$.

Cost: Since each party P_i sends s_i and π_i to every other party, the communication cost is $O(n^2 \cdot (|v| + \lambda))$ bits. Π_{RBC} incurs $O(1)$ asynchronous rounds.

Fig. 9: Code for party $P_i \in \mathcal{P}$ having input v_i to agree on a common subset $\mathbf{v}$ of inputs from $n - t$ parties.

D Agreement on a Common Subset

In this section, we present our Agreement on a Common Subset (ACS) protocol Π_{ACS} that outputs a common set $\mathbf{v}$ of inputs provided by parties, of which at least $n - 2t$ are provided by *honest* parties. Each input additionally satisfies an externally valid view-based predicate Q . An overview of our protocol was provided in § 7.2. Here, we present detailed steps of Π_{ACS} in Figure 9 and formally prove its correctness.

Π_{ACS} uses n parallel instances of Π_{EVBA} to decide which inputs to consider in the output. Each of these instances uses a common coin, the setup for which is instantiated *during* Π_{ACS} . A common coin can be generated using the Diffie-Helman threshold coin tossing protocol [CKS00] that assumes that a key s has been secret shared among the parties, with g^s and g^{s_i} being public, where s_i is P_i 's share of s . Here, we hold each P_i responsible for generating this setup for $\Pi_{\text{EVBA}}^{(i)}$.

To do this, each P_j is made to sample and secret share a uniformly random value s_j . P_i identifies set of setup_i $t + 1$ parties who have secret shared these values and broadcasts setup_i along with v_i . Each party's share for the coin for $\Pi_{\text{EVBA}}^{(i)}$ is obtained by summing up the secret shares obtained from parties in setup_i : this sum is guaranteed to be uniformly random since one value is generated by an honest party.

Note that a corrupt P_i may not broadcast setup_i at all. To ensure that $\Pi_{\text{EVBA}}^{(i)}$ still terminates, we use the fact that Π_{EVBA} is *Good-Case-Coin-Free*, i.e, if only 0 is externally valid, then $\Pi_{\text{EVBA}}^{(i)}$ terminates even *without* a common coin.

We proceed to prove the properties of Π_{ACS} . We first prove that the view-based predicates used in the Π_{EVBA} instances have eventual validation.

Claim. For each $P_j \in \mathcal{P}$, if Q is a view-based predicate, then $Q^{(j)}$ defined in Figure 9 is a view-based predicate.

Proof. Suppose that $Q^{(j)}(\text{view}_i, 1)$ is true for some party P_i . This means that P_i has received setup_j from $\Pi_{\text{RBC}}^{(j)}$ of size at least $n - t$, and a secret share from each party in setup_j . P_i has also received v_j for which $Q(\text{view}_i, v_j)$ holds.

The termination and consistency of $\Pi_{\text{RBC}}^{(j)}$ ensures that every other party P_k also receives the same setup_j from $\Pi_{\text{RBC}}^{(j)}$. The termination of Π_{VSS} guarantees that P_k also obtains a secret share from each party in setup_j . Since Q is a view-based predicate, each P_k eventually has a view view_k where $Q(\text{view}_k, v_j)$ holds, implying that $Q^{(j)}(\text{view}_k, 1)$ holds true.

From the above argument, we can also conclude that if the size of CORE grows to $n - t$ in P_i 's view, then it also eventually grows to $n - t$ in P_k 's view. Thus, if $Q^{(j)}(\text{view}_i, 0)$ is true for P_i , then every P_k eventually has a view view_k for which $Q(\text{view}_k, 0)$ is true. $\square$

Lemma 26. *Suppose that each $P_i \in \mathcal{P}$ has input v_i . Protocol Π_{ACS} in an Agreement on Common Subset protocol has the following properties.*

- *Validity: If an honest party outputs a set $\mathbf{v}$, then $|\mathbf{v}| \geq n - t$ and $\mathbf{v}$ contains the inputs of at least $n - 2t$ honest parties. For each $v \in \mathbf{v}$, $Q(v)$ holds.*
- *Agreement: If an honest party outputs $\mathbf{v}$, then every honest party outputs $\mathbf{v}$.*
- *Termination: If $n - t$ honest parties participate with an input, then all honest parties output $\mathbf{v}$.*

Proof. Validity follows from the external validity of Π_{EVBA} , since the output 0 is valid only if at least $n - t$ instances of Π_{EVBA} output 1. Agreement follows from the agreement of Π_{EVBA} . To prove that termination is guaranteed, we prove that if all honest parties participate with an input, then at least $n - t$ instances of Π_{EVBA} terminate with output 1 for some honest party P_i . From the termination of Π_{EVBA} , this implies that these $n - t$ instances terminate with output 1 for *every* honest party. Consequently, all parties participate in *all* instances of Π_{EVBA} with an externally valid input, and from the termination of Π_{EVBA} , all parties output from all Π_{EVBA} instances.

For contradiction, suppose that for all honest parties, fewer than $n - t$ instances of Π_{EVBA} terminate with output 1. Note that each honest P_j broadcasts (v_j, setup_j) . The validity of Π_{RBC} and termination of Π_{VSS} ensure that P_i eventually receives (v_j, setup_j) , and receives an output from $\Pi_{\text{VSS}}^{(k)}$ for each $P_k \in \text{setup}_j$. Since $n - t$ instances of Π_{EVBA} have not terminated yet, P_i participates in $\Pi_{\text{EVBA}}^{(j)}$ with input 1. Further, note that only $Q^{(j)}(\text{view}, 1)$ is valid, where view is the view of *any* honest party. Thus, $\Pi_{\text{EVBA}}^{(j)}$ terminates with output 1 for each honest P_j , implying that $n - t$ instances of Π_{EVBA} terminate for P_i . $\square$

E Multi Party Computation

In this section, we present our protocols for MPC. Our MPC protocol includes inputs from a synchronously extended set of parties of size at least $n - t$. We first present a lower bound to show that this is the best that can be achieved. We then present our protocols that follow the circuit evaluation paradigm that securely evaluates a circuit of linear and multiplication gates that represents the function F to be computed: starting with secret shares of the parties' inputs, given secret shares of inputs of a gate, we present protocols to compute secret shares of the output of the gate.

Linear gates are easier to compute due to the linearity of our secret sharing scheme. The bulk of this section is dedicated to securely computing multiplication gates. In the rest of the section, we use c_M and D_M to respectively denote the number of multiplication gates and the multiplicative depth of the circuit.

E.1 Lower Bound for Inputs Considered

Due to the nature of the granular synchrony model, it is not possible to achieve an MPC protocol that incorporates the inputs of each honest party. Here, we prove a lower bound on the number of inputs that can be considered for MPC. Following [BLZL20], we measure this by a protocol's *output quality*:

Definition 16 (Output quality). *The Output Quality of an execution of any protocol Π that evaluates a function F is L if parties output $F(x_1, \dots, x_n)$, where the number of inputs x_i that are not $\perp$ is at least L .*

Asynchronous MPC protocols achieve $n - t$ output quality. We prove that, in granular synchrony, even when *all* parties are honest, the best output quality that can be achieved is the minimum size of any *synchronously extended* set of $n - t$ parties. Our proof is inspired by a proof given in [BLZL20].

Lemma 27. *For every MPC protocol Π with guaranteed output delivery tolerating up to t corruptions, there exists a network configuration satisfying Condition 1 for which every execution of Π achieves an output quality equal to the minimum size of a synchronously extended set of size at least $n - t$, even when all parties are honest.*

Proof. Without loss of generality, suppose that the function to be computed is the OR function. Let $S_1 \subseteq \mathcal{P}$ be the smallest synchronously extended set of parties of size at least $n - t$. For contradiction, suppose Π always considers more than $|S_1|$ inputs. Let S_2 be $\mathcal{P} \setminus S_1$. Note that $|S_2| \leq t$. Now, consider the following execution of Π :

Execution 1: All parties in S_1 are honest and participate with input 0. All parties in S_2 are corrupt, and they immediately abort.

Because Π achieves guaranteed output delivery and because all inputs are 0, each party in S_1 must eventually output 0 by some point in time, say time T . Contrast that with the following:

Execution 2: All parties in S_1 are honest and participate with input 0. All parties in S_2 are honest and participate with input 1. The adversary delays messages sent between S_1 and S_2 by at least T time.

From the perspective of a party in S_1 and before time T , the two executions appear the same: each party in S_1 receives no message from any party in S_2 . Since parties in S_1 output 0 in Execution 1, they must also output 0 in Execution 2. However, we assumed Π incorporates more than $|S_1|$ inputs. Thus, Π must include the input of at least one party in S_2 , and in Execution 2 each such input is 1. By correctness of Π and by the fact that the parties compute OR, parties in S_1 must output 1 in Execution 2 – a contradiction. $\square$

E.2 Beaver Triples

Consider a multiplication gate with inputs x and y . Given $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$, Beaver's trick uses secret shares of a so-called Beaver triple $(\llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)$ to compute secret shares of the output of the multiplication gate i.e $\llbracket xy \rrbracket$. Here, a and b are uniformly random, and it is assured that $c = ab$. Thus, a and b can be used to reconstruct masked versions d and e of x and y , and $z = xy$ is computed as a linear combination of $\llbracket a \rrbracket$, $\llbracket b \rrbracket$ and $\llbracket c \rrbracket$. We present Π_{Beaver} in Figure 10.

Lemma 28. *Let $\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket$ be inputs to Π_{Beaver} . If $c = ab$ holds, then the parties output $\llbracket z \rrbracket = \llbracket xy \rrbracket$. If a and b are uniformly random, then the view of $\mathcal{A}$ is independent of x and y . Π_{Beaver} incurs a communication of $O(n^2 \cdot (\log |\mathbb{F}| + \lambda))$ bits and $O(1)$ asynchronous rounds.*

Protocol $\Pi_{\text{Beaver}}(\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)$

- Locally compute $\llbracket d \rrbracket = \llbracket x \rrbracket - \llbracket a \rrbracket$ and $\llbracket e \rrbracket = \llbracket y \rrbracket - \llbracket b \rrbracket$. Execute $\Pi_{\text{Rec}}(\llbracket d \rrbracket)$ and $\Pi_{\text{Rec}}(\llbracket e \rrbracket)$ to obtain d and e .
- Locally compute and output $\llbracket z \rrbracket = de + d\llbracket b \rrbracket + e\llbracket a \rrbracket + \llbracket c \rrbracket$.

Fig. 10: Computing $\llbracket xy \rrbracket$ given a uniformly random multiplication triple $(\llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)$

Proof. xy can be written as $(x - a + a)(y - b + b)$. If $d = x - a$ and $e = y - b$, then $z = (d + a)(e + b) = de + db + ea + ab$. If $\llbracket c \rrbracket = \llbracket ab \rrbracket$, then $\llbracket z \rrbracket = de + d\llbracket b \rrbracket + e\llbracket a \rrbracket + \llbracket c \rrbracket$.

The view of $\mathcal{A}$ consists of d and e , along with shares of the inputs. From the privacy of Π_{VSS} , $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$ are independent of x and y . Since a and b are uniformly random, even if $\mathcal{A}$ learns $d = x - a$ and $e = y - b$, a and b mask x and y . Thus, $\mathcal{A}$'s view is independent of x and y . The communication cost and round complexity follow from that of Π_{Rec} . $\square$

E.3 Verifiable Triple Generation

In this section, we present protocol Π_{TripSh} that identifies a subset $\mathcal{CS}$ of parties that have secret shared c_M multiplication triples. Π_{ASP} is used to identify a subset $\mathcal{CS}$ of parties, where each $P_i \in \mathcal{CS}$ has shared c_M triples $\{(a_\ell^i, b_\ell^i, c_\ell^i)\}_{\ell \in [c_M]}$. To check that $c_\ell^i = a_\ell^i b_\ell^i$ holds, each party additionally shares another batch of c_M triples $\{(x_\ell^i, y_\ell^i, z_\ell^i)\}_{\ell \in [c_M]}$. By checking that $r(a_\ell^i b_\ell^i - c_\ell^i) = z_\ell^i - x_\ell^i y_\ell^i$ holds for some uniformly random r that is unknown to P_i before the triples were shared, parties verify that $(a_\ell^i, b_\ell^i, c_\ell^i)$ is indeed a multiplication triple. We present Π_{TripSh} in Figure 11.

Protocol $\Pi_{\text{TripSh}}()$

- Choose values $\{(a_\ell^i, b_\ell^i, c_\ell^i)\}_{\ell \in [c_M]}$ and $\{(x_\ell^i, y_\ell^i, z_\ell^i)\}_{\ell \in [c_M]}$ such that $a_\ell^i, b_\ell^i, x_\ell^i, y_\ell^i$ are chosen uniformly at random and that $c_\ell^i = a_\ell^i b_\ell^i$ and $z_\ell^i = x_\ell^i y_\ell^i$ for each $\ell \in [c_M]$. Additionally, choose r_i uniformly at random.
- Participate in $\Pi_{\text{ASP}}(6c_M + 1, \{a_\ell^i, b_\ell^i, c_\ell^i, x_\ell^i, y_\ell^i, z_\ell^i\}_{\ell \in [c_M]} \cup \{r_i\})$ to obtain output $\mathcal{CS}$.
- Locally compute $\llbracket r \rrbracket = \sum_{P_j \in \mathcal{CS}} \llbracket r_j \rrbracket$. Participate in $\Pi_{\text{Rec}}(\llbracket r \rrbracket)$ to obtain r .
- For each $P_j \in \mathcal{CS}$ and $\ell \in [c_M]$, compute $\llbracket \rho_\ell^j \rrbracket = r\llbracket a_\ell^j \rrbracket - \llbracket x_\ell^j \rrbracket$ and $\llbracket \sigma_\ell^j \rrbracket = \llbracket b_\ell^j \rrbracket - \llbracket y_\ell^j \rrbracket$. Participate in $\Pi_{\text{Rec}}(\llbracket \rho_\ell^j \rrbracket)$ and $\Pi_{\text{Rec}}(\llbracket \sigma_\ell^j \rrbracket)$ to obtain ρ_ℓ^j and σ_ℓ^j .
- For each $P_j \in \mathcal{CS}$ do the following.
 - For each $\ell \in [c_M]$, compute $\llbracket \tau_\ell^j \rrbracket = r_\ell^j \llbracket c_\ell^j \rrbracket - \llbracket z_\ell^j \rrbracket - \sigma_\ell^j \llbracket x_\ell^j \rrbracket - \rho_\ell^j \llbracket y_\ell^j \rrbracket - \rho_\ell^j \sigma_\ell^j$.
 - For each $\ell \in [c_M]$, participate in $\Pi_{\text{Rec}}(\llbracket \tau_\ell^j \rrbracket)$ to obtain τ_ℓ^j . If $\tau_\ell^j \neq 0$, remove P_j from $\mathcal{CS}$.
- On computing τ_ℓ^j for each $P_j \in \mathcal{CS}$ such that $\tau_\ell^j = 0$ for all $\ell \in [c_M]$, output $\mathcal{CS}, \{(\llbracket a_\ell^j \rrbracket, \llbracket b_\ell^j \rrbracket, \llbracket c_\ell^j \rrbracket)\}_{P_j \in \mathcal{CS}, \ell \in [c_M]}$.

Fig. 11: Protocol to identify a common set $\mathcal{CS}$ of parties that have secret-shared c_M multiplication triples.

Lemma 29. *At the end of Π_{TripSh} , each party outputs $\mathcal{CS}$ and $\{(\llbracket a_\ell^j \rrbracket, \llbracket b_\ell^j \rrbracket, \llbracket c_\ell^j \rrbracket)\}_{P_j \in \mathcal{CS}, \ell \in [c_M]}$ such that $c_\ell^j = a_\ell^j b_\ell^j$ holds for each $P_j \in \mathcal{CS}$ and $\ell \in [c_M]$, except with negligible probability, where $\mathcal{CS}$ contains a synchronously extended set of size at least $n - 2t$. If P_j is honest, then $\mathcal{A}$'s view is independent of $(a_\ell^j, b_\ell^j, c_\ell^j)$. The protocol incurs a communication of $O(n^4 \cdot (c_M \log |\mathbb{F}| + n^2 \lambda))$ bits, and a round complexity of $O(D \log n)$ synchronous and $O(\log n)$ asynchronous rounds in expectation.*

Proof. We first argue that $a_\ell^j b_\ell^j = c_\ell^j$ holds for all $P_j \in \mathcal{CS}$ and $\ell \in [c_M]$, except with probability $\frac{tc_M}{|\mathbb{F}|}$. From the correctness of Π_{ASP} , $\mathcal{CS}$ contains a synchronously extended set of size at least $n - 2t$. For each $P_j \in \mathcal{CS}$ where P_j is honest, $a_\ell^j b_\ell^j = c_\ell^j$ holds since P_j chooses c_ℓ^j to this way. Thus, these parties are not removed from $\mathcal{CS}$. We prove that, except with probability $\frac{1}{|\mathbb{F}|}$, this holds for corrupt $P_j \in \mathcal{CS}$ too. Note that $\tau_\ell^j = 0$. Thus,

$$rc_\ell^j - z_\ell^j - \sigma_\ell^j x_\ell^j - \rho_\ell^j y_\ell^j - \sigma_\ell^j \rho_\ell^j = 0$$

Substituting $\rho_\ell^j = ra_\ell^j - x_\ell^j$ and $\sigma_\ell^j = b_\ell^j - y_\ell^j$,

$$rc_\ell^j - z_\ell^j - (b_\ell^j - y_\ell^j)x_\ell^j - (ra_\ell^j - x_\ell^j)y_\ell^j - (ra_\ell^j - x_\ell^j)(b_\ell^j - y_\ell^j) = 0$$

Note that the $(ra_\ell^j - x_\ell^j)y_\ell^j$ terms cancel out. Thus,

$$rc_\ell^j - z_\ell^j - (b_\ell^j - y_\ell^j)x_\ell^j - (ra_\ell^j - x_\ell^j)b_\ell^j = 0$$

$$\implies rc_\ell^j - z_\ell^j + y_\ell^j x_\ell^j - ra_\ell^j b_\ell^j = 0$$

$$\implies r(c_\ell^j - a_\ell^j b_\ell^j) = z_\ell^j - x_\ell^j y_\ell^j$$

If $c_\ell^j \neq a_\ell^j b_\ell^j$, then $\tau_\ell^j = 0$ only when $r = (z_\ell^j - x_\ell^j y_\ell^j)(c_\ell^j - a_\ell^j b_\ell^j)^{-1}$. Note that r is uniformly random. This is because $r = \sum_{P_k \in \mathcal{CS}} r_k$, and at least one honest party $P_h \in \mathcal{CS}$ chose r_h uniformly at random. Since r is chosen uniformly at random, this occurs with probability $\frac{1}{|\mathbb{F}|}$. Taking a union bound, $c_\ell^j = a_\ell^j b_\ell^j$ holds for all parties in $\mathcal{CS}$ and each $\ell \in [c_M]$ except with probability $\frac{tc_M}{|\mathbb{F}|}$.

The view of $\mathcal{A}$ consists of r , along with ρ_ℓ^j and σ_ℓ^j for each $P_j \in \mathcal{CS}$ and $\ell \in [c_M]$. This is because the privacy of Π_{VSS} guarantees that $\mathcal{A}$'s view is independent of values that are secret-shared. Now, consider honest $P_j \in \mathcal{CS}$. Since P_j chooses x_ℓ^j, y_ℓ^j uniformly at random, ρ_ℓ^j and σ_ℓ^j are also uniformly random. Thus, the view of $\mathcal{A}$ is independent of $(a_\ell^j, b_\ell^j, c_\ell^j)$.

The communication cost and round complexity follow from the cost of invoking Π_{ASP} to share $O(c_M)$ values, which incurs a communication of $O(n^4 \cdot (c_M \log |\mathbb{F}| + n^2 \lambda))$ bits. $\square$

E.4 The Offline Pre-processing phase

Π_{Prep} is used to generate c_M beaver triples in an input-independent pre-processing phase. Using Π_{TripSh} , a set $\mathcal{CS}$ of parties who have verifiably secret shared multiplication triples can be identified. To obtain triples that are *uniformly random*, for each $\ell \in [c_M]$, the shared triples $\{(\llbracket a_\ell^j \rrbracket, \llbracket b_\ell^j \rrbracket), \llbracket c_\ell^j \rrbracket\}_{P_j \in \mathcal{CS}}$ are used to define transformed triples $\{(\llbracket \mathbf{a}_\ell^j \rrbracket, \llbracket \mathbf{b}_\ell^j \rrbracket), \llbracket \mathbf{c}_\ell^j \rrbracket\}_{P_j \in \mathcal{CS}}$ that lie on polynomials $A_\ell(\cdot), B_\ell(\cdot)$ and $C_\ell(\cdot)$ of degrees t, t and $2t$ respectively. A multiplication triple that is unknown to $\mathcal{A}$ can be sampled by evaluating these polynomials at a point that was not evaluated previously.

To be able to define polynomial $C_\ell(\cdot)$ of degree $2t$, we need $\mathcal{CS}$ to be of size at least $2t + 1$. Following a trick used in [BCV24], we add dummy triples to $\mathcal{CS}$ until it is $2t + 1$ in size. Then, we define polynomials $A(\cdot), B(\cdot)$ and $C(\cdot)$ as follows. Suppose that $\mathcal{CS} = \{P_1, \dots, P_{2t+1}\}$. The first $t + 1$ points $\{(\llbracket a_\ell^j \rrbracket, \llbracket b_\ell^j \rrbracket)\}_{j \in [t+1]}$ are used to define $A_\ell(\cdot)$ and $B_\ell(\cdot)$. Consequently, the first $t + 1$ transformed points $\{(\llbracket \mathbf{a}_\ell^j \rrbracket, \llbracket \mathbf{b}_\ell^j \rrbracket), \llbracket \mathbf{c}_\ell^j \rrbracket\}_{j \in [t+1]}$ equal $\{(\llbracket a_\ell^j \rrbracket, \llbracket b_\ell^j \rrbracket)\}_{j \in [t+1]}$. Using these, parties can

locally compute the remaining t points $\{(\llbracket \mathbf{a}_\ell^j \rrbracket, \llbracket \mathbf{b}_\ell^j \rrbracket)\}_{j \in (t+1, 2t+1]}$ through polynomial interpolation. The points $\{\llbracket \mathbf{c}_\ell^j \rrbracket\}_{j \in (t+1, 2t+1]}$ points can be computed using Beaver's trick.

Note that Beaver's trick ensures that the adversary learns a transformed triple only if it knows the triple that is being transformed. Since the number of honest parties in $\mathcal{CS}$ is of size at least $t+1$, even after including transformed dummy triples, $\mathcal{A}$ learns at most t transformed triples. We present Π_{Prep} in Figure 12.

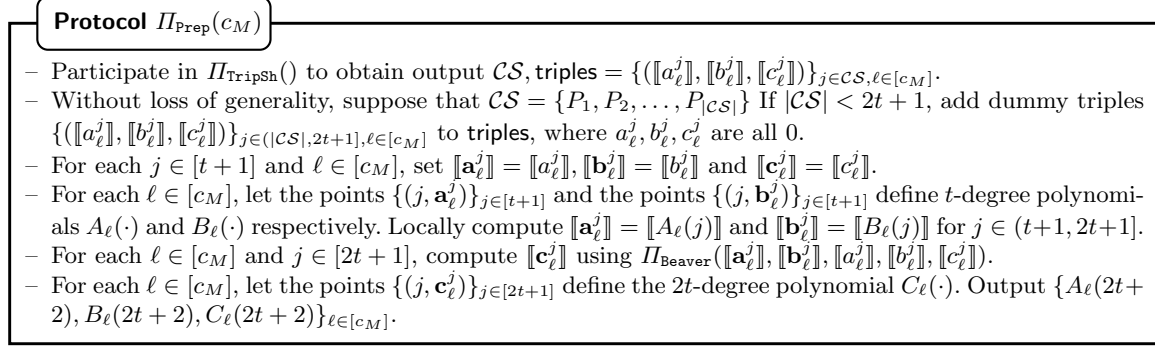


Fig. 12: Code for party P_i to obtain c_M secret-shared multiplication triples

Lemma 30. *At the end of Π_{Prep} , the following hold except with negligible probability. 1) Each party outputs $\{(\llbracket a_\ell \rrbracket, \llbracket b_\ell \rrbracket, \llbracket c_\ell \rrbracket)\}_{\ell \in [c_M]}$ such that for each $\ell \in [c_M]$, $c_\ell = a_\ell b_\ell$ holds. 2) $\mathcal{A}$'s view is independent of (a_ℓ, b_ℓ, c_ℓ) . The protocol incurs a communication of $O(n^4 \cdot (c_M \log |\mathbb{F}| + n^2 \lambda))$ bits, and a round complexity of $O(D \log n)$ synchronous and $O(\log n)$ asynchronous rounds in expectation.*

Proof. Consider any $\ell \in [c_M]$. We prove that the polynomials $A_\ell(\cdot), B_\ell(\cdot)$ and $C_\ell(\cdot)$ are computed such that $C_\ell(\cdot) = A_\ell(\cdot) \cdot B_\ell(\cdot)$ holds. We additionally prove that $\mathcal{A}$ learns at most t points on any of these polynomials and thus, $\mathcal{A}$'s view is independent of $A_\ell(2t+2), B_\ell(2t+2), C_\ell(2t+2)$. This implies that $c_\ell = a_\ell b_\ell$ holds, and $\mathcal{A}$'s view is independent of (a_ℓ, b_ℓ, c_ℓ) , completing the proof.

To prove that $C(\cdot)_\ell = A(\cdot)_\ell \cdot B(\cdot)_\ell$, since $C(\cdot)$ is of degree $2t$, it is enough to show that $C(j) = A(j) \cdot B(j)$ for $j \in [2t+1]$. From Lemma 29, $c_\ell^j = a_\ell^j \cdot b_\ell^j$ holds for each $(\llbracket a_\ell^j \rrbracket, \llbracket b_\ell^j \rrbracket, \llbracket c_\ell^j \rrbracket) \in \text{triples}$. Thus, from Lemma 28, $\llbracket \mathbf{c}_\ell^j \rrbracket = \llbracket \mathbf{a}_\ell^j \rrbracket \cdot \llbracket \mathbf{b}_\ell^j \rrbracket$ holds for each $j \in [2t+1]$.

From Lemma 29, $\mathcal{CS}$ a synchronously extended set of size at least $n - 2t$. By following the same reasoning used to argue that *honest majority* holds for a collection, $\mathcal{CS}$ contains at least $t+1$ honest parties. For each $(\llbracket a_\ell^j \rrbracket, \llbracket b_\ell^j \rrbracket, \llbracket c_\ell^j \rrbracket) \in \text{triples}$, from Lemma 28, $\mathcal{A}$'s view is independent of $(\llbracket \mathbf{a}_\ell^j \rrbracket, \llbracket \mathbf{b}_\ell^j \rrbracket, \llbracket \mathbf{c}_\ell^j \rrbracket)$ for honest $P_j \in \mathcal{CS}$. Since **triples** is of size exactly $2t+1$, this means that $\mathcal{A}$ learns at most t points on $A(\cdot), B(\cdot)$ and $C(\cdot)$.

The communication cost and round complexity follow from the cost of Π_{TriPSH} . □

E.5 The online phase

In this section we present the input dependent circuit evaluation protocol Π_{CircEval} . The pre-processing phase protocol Π_{Prep} is run to generate c_M multiplication triples. On obtaining their inputs, parties secret share these using Π_{ASP} . Starting from input gates, the circuit is then evaluated in a topological order. Shares of the outputs of linear gates are locally evaluated, and shares of the

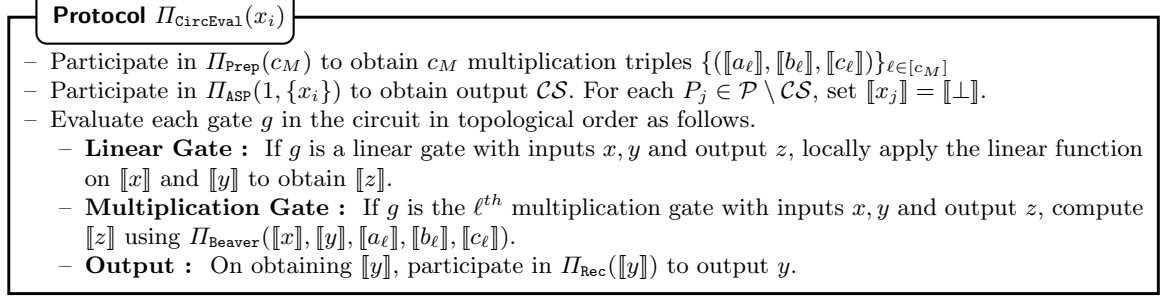


Fig. 13: Code for party P_i having input x_i to securely evaluate a function F on a common set of parties' inputs

outputs of multiplication gates are evaluated using Π_{Beaver} . Finally, Π_{Rec} is used to reconstruct the output. We present Π_{CircEval} in Figure 13.

The following is immediate from Lemmas 28 and 30.

Theorem 1. *Each P_i participates in Π_{CircEval} to achieve the following, except with negligible probability. (1) Each P_i outputs a set $\mathcal{CS}$ that contains a synchronously extended set of size at least $n - t$, and a value $y = F(x_1, \dots, x_n)$, where $x_j = \perp$ if $P_j \notin \mathcal{CS}$. (2) The view of $\mathcal{A}$ is independent of x_i for each honest P_i . Π_{CircEval} incurs a communication of $O(n^4 \cdot (c_M \log |\mathbb{F}| + n^2 \lambda))$ and $O(n^2 \lambda \cdot c_M)$ bits in the offline and online phases, respectively, and a round complexity of $O(D \log n)$ synchronous and $O(\log n)$ asynchronous rounds in expectation.*